



국민대학교
소프트웨어융합대학
소프트웨어학부

다학제간캡스톤디자인 종합설계프로젝트

프로젝트 명	TraceInspector - 추적 가능한 시각화 기반 코드 정적분석기
팀 명	팀 2
문서 제목	최종 보고서

날짜	2026-06-16
----	------------

팀원	김신영 (조장)
	강민성
지도교수	김형균

제 출 문

본 보고서를 다학제간캡스톤디자인I 교과목의
종합설계 프로젝트 최종보고서로 제출합니다.

2026.06.19

프로젝트 명 : TraceInspector

팀 구성원 : 김신영(팀장)
강민성

지도교수 : 김형균 교수

목 차

1. 요약
2. 중간 보고서 발표자료
3. 결과 보고서
 - 3.1 결과 보고서
 - 3.2 결과 보고서 발표자료
4. 멘토링 보고서
5. 회의록

제 1 장

요약

코드 정적 분석은 프로그램을 실행하지 않고 코드를 논리적으로 분석하여 동적 특성을 파악하는 기법이며 런타임 오류, 주어진 명세 준수 등 다양한 논리적/의미적 성질을 검증하기 위해 사용된다. TracInspector는 요약해석(abstract interpretation)이론을 바탕으로 구현된 추적 가능한 시각화 기반 코드 정적분석기이다. 단순히 오류를 탐지해서 알려주는 정적 분석기를 넘어, 분석 과정을 시각적으로 나타내어 사용자가 어떤 과정을 통해 오류가 탐지되었는지 해석 가능하게끔 돕고자 한다.

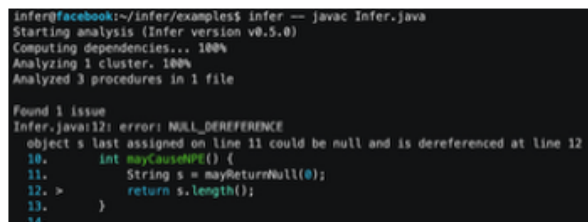
제 2 장

중간 보고서 발표자료

정적 프로그램 분석을 활용한 코드 오류 탐지 및 시각화

정적 프로그램 분석이란?

- 프로그램을 직접 실행하지 않고 동작 특성 및 오류를 탐지하는 기술
- 각종 런타임 오류를 미리 탐지 가능!
- 전용 소프트웨어 및 IDE 등에 탑재



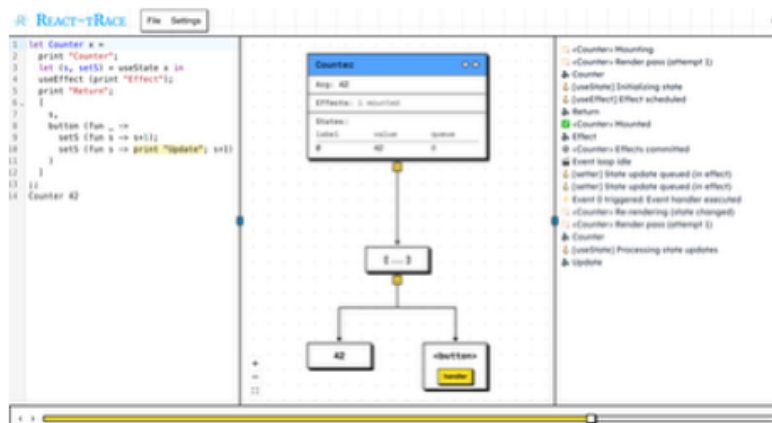
프로젝트 배경 및 동기

- 기존 정적 분석기는 분석 속도, 사용자 편의성, 정확성에 초점
- 정적 분석 기법은 결과뿐만 아니라 분석 과정 및 근거에 대한 이해가 필요
- 프로그램 분석 흐름을 사용자가 투명하게 따라갈 수 있는 서비스 개발
- 분석 속도, 지원 기능 범위보단 직관성, 시각화 등 “교육적 목표”

3

프로젝트 배경 및 동기

- 기존의 정적 분석 도구와 다른 지향점 - 시각화 및 분석 흐름 이해
- React-tRace: React hook 랜더링 의미론 및 시각화 연구



React+Trace: A Semantics for Understanding React Hooks: An Operational Semantics and a Visualizer for Clarifying React Hooks, J. Lee et al., OOPSLA 2025
<https://dl.acm.org/doi/10.1145/3763067>

4

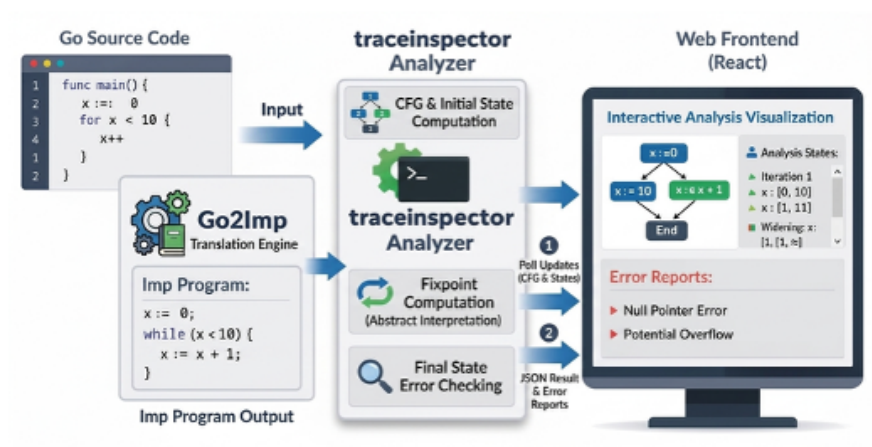
TraceInspector 소개

- 정적 분석을 이용한 프로그램 오류 탐지 및 분석 시각화 서비스
- 이론적 배경: 요약해석(abstract interpretation)
 - 프로그램의 구체적 실행값이 아닌 추상화된 근사값을 계산하는 기법
 - 신뢰 가능한 엄밀한 분석 결과 도출 가능
- 서비스 목표
 - 입력 프로그램 소스코드 요약실행 및 해석을 통한 런타임 오류 탐지
 - 분석 알고리즘 시각화 및 동작 방식, 흐름 전달

5

TraceInspector 서비스 구조 소개

- 분석 프레임워크를 구현한 분석기 프로그램
- 소스 입력 및 결과 시각화를 위한 웹 인터페이스



6

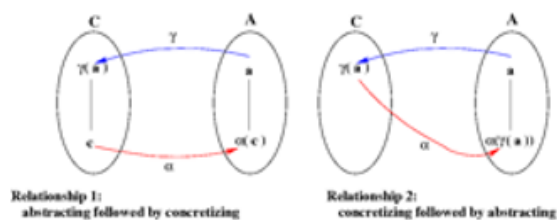
TraceInspector 분석기 소개

- 정적 분석(요약해석) 알고리즘 구현체
 - 단순히 분석기를 작성한게 아닌 분석기의 정확성에 대한 이론적 보장
- Go언어로 작성
 - 분석이 수월한 간결한 문법
 - Go 코드 처리를 위한 훌륭한 stdlib(parsing, AST, typecheck, etc.)
- 분석 대상 입력 언어: Imp (**S**imple **I**mperative Language)
 - 자체 구현 - 분석하기 편리한 형태로 언어 및 형식의미론 설계
 - int, bool, arrays, if-else, while, I/O, functions
 - TraceInspector에 Go -> Imp번역기 탑재
 - 타 언어 -> Imp 번역기 구현 시 **입력 언어 무관** 분석 가능

7

요약해석(abstract interpretation) 소개

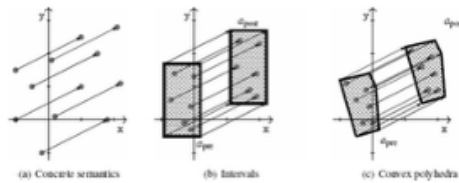
- 정지 문제/Rice Theorem: 오류가 있는지 자동으로 완벽하게 맞추는 분석기는 이론상 **불가능**
- 완벽을 포기: 모든 오류를 잡을 수 없지만, 오류가 있다고 판정하면 실제로 오류가 발생하는 soundness가 만족하도록 설계
- 프로그램 실행 의미론 보수적인 근사를 취하여 안전한 결과를 보장
- soundness가 만족됨이 이론적으로 증명됨(galois connection)



8

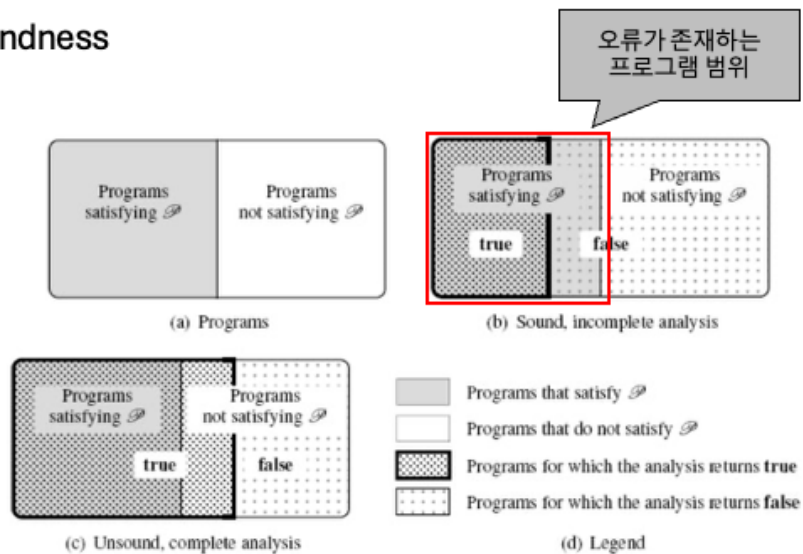
요약해석을 이용한 오류 탐지

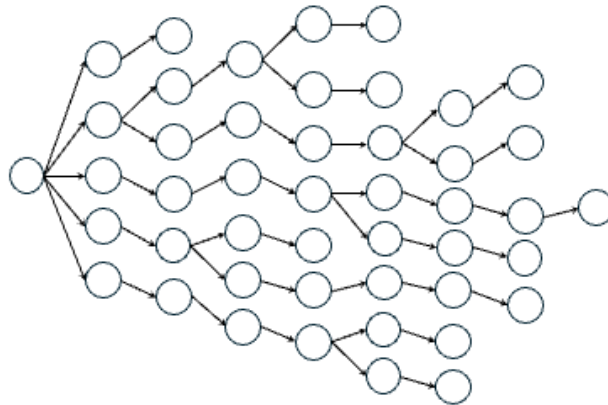
- 프로그램 내 단계별 상태(변수의 값 등)를 근사
- 근사를 위한 요약의미론 및 요약도메인 구현
 - 정수값의 부호 (음수, 0, 양수), 구간($v = [n, m] \rightarrow n \leq v \leq m$)
 - 복합적인 조건식을 위한 관계형 도메인 ($v > m \rightarrow v > m \vee v \leq m$)
- 요약도메인 정보를 활용한 오류 탐지
 - Divide by zero error – 부호 도메인 값 활용
 - Array out of bounds indexing – 배열 index, 길이에 대한 구간값 활용
 - Unreachable code – 주어진 요약상태에서 관계형 도메인의 진리값 판별



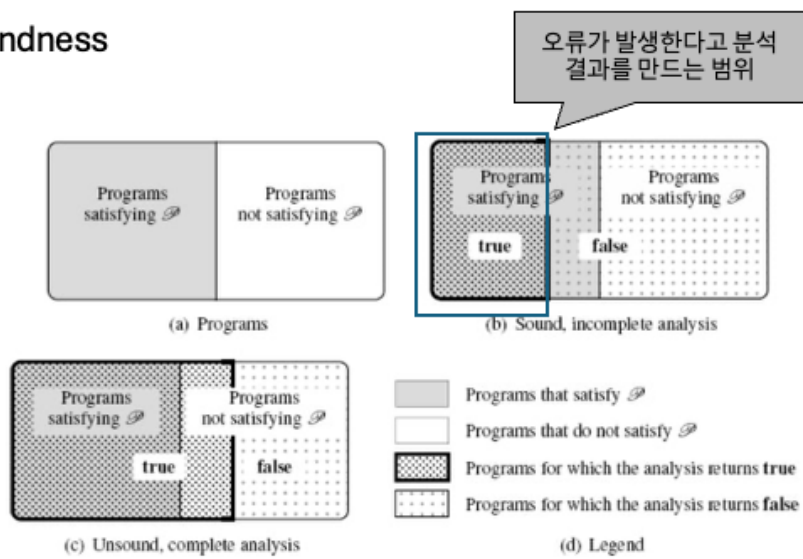
9

Soundness

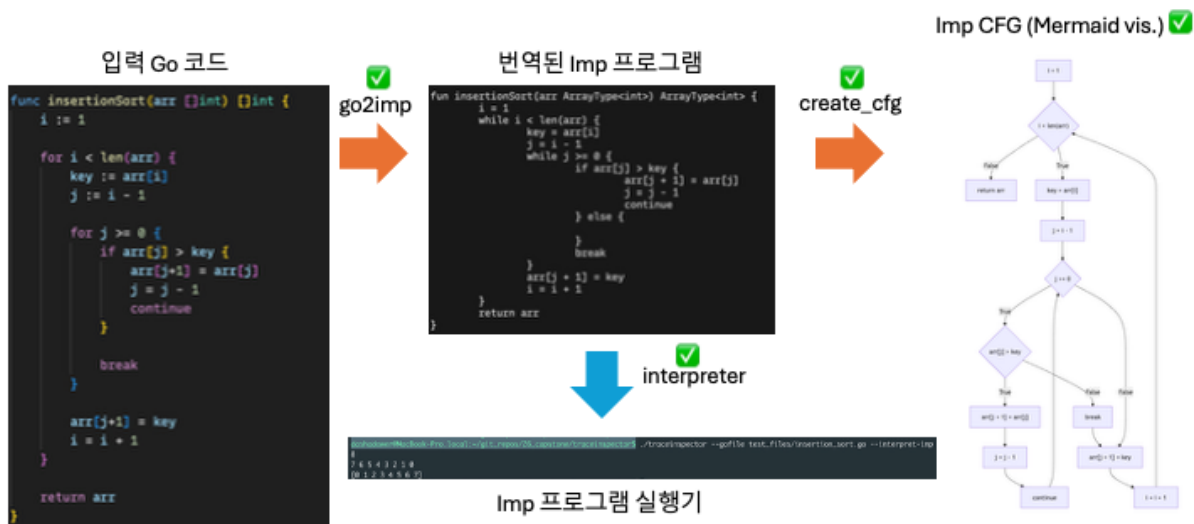




Soundness



TracInspector 분석기 구현 현황

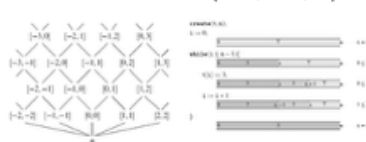


13

TracInspector 분석기 향후 계획

Imp CFG (Mermaid vis.)

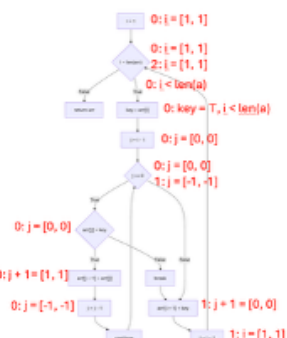
Abstract Domain(요약도메인)



Abstract Interpreter(요약실행기)

Abstract Semantics(요약의미론)

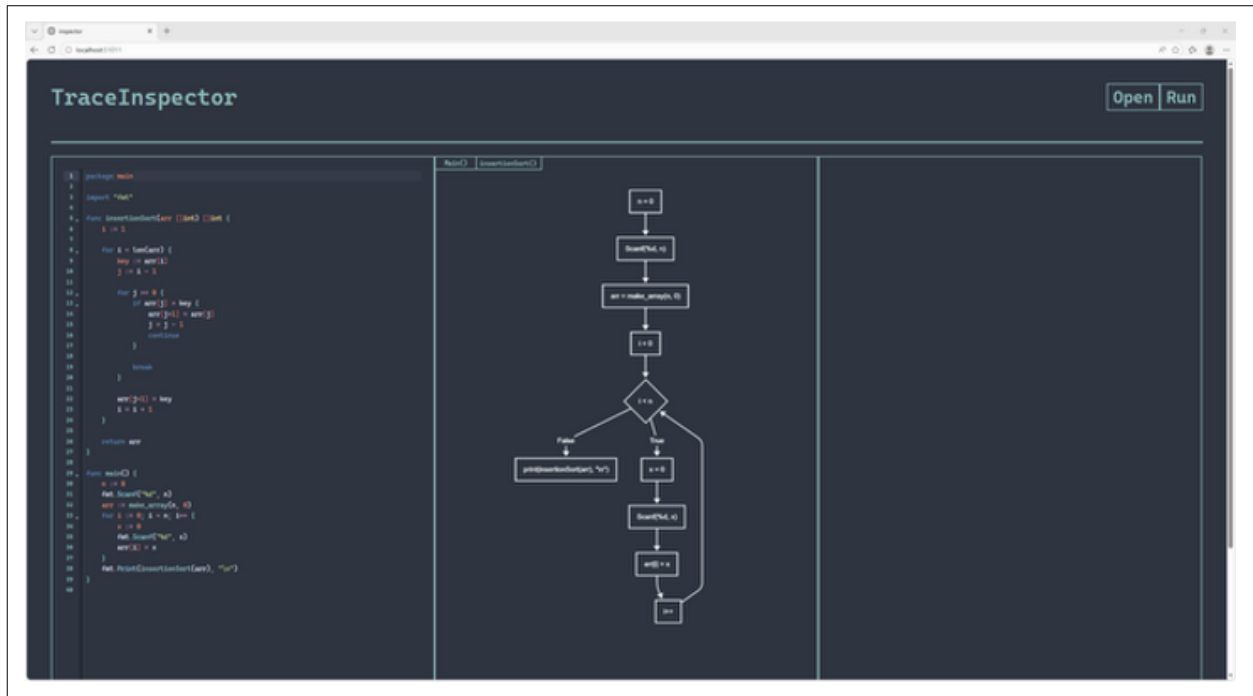
Abstract State(요약상태)



Safety Property(오류 없는 실행 조건 검증)

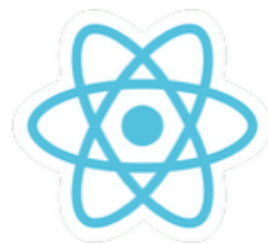
$\forall \sigma, called\{arr[e], \sigma\} \rightarrow [[e]]_{\sigma}^{\#} \geq 0 \wedge [[e]]_{\sigma}^{\#} \leq len_{\sigma}^{\#}(arr)$

14



사용된 라이브러리

- 현 상태 제작을 위해 다음과 같은 것들이 활용됨
- 기본적인 언어는 TypeScript 언어가 사용됨
 - 패키지 관리를 위해 npm + NodeJS가 사용됨
 - 프레임워크는 Vite + React가 사용됨
 - 코드 편집기 제작은 @codemirror가 사용됨
 - 그래프 제작은 @mermaid가 사용됨
 - 파일시스템 접근을 위해 express와 함께 사용됨



현재 피드백

- 디자인이 너무 심플하다는 의견을 받아, 추후에 배너 같은 것들을 도입할 계획
- 각 상태 박스를 조정할 수 있어야 한다는 의견을 받아, 관련 라이브러리를 도입할 계획
- 추후 제작될 그래프 관련 애니메이션 기능을 삽입해야 한다는 의견을 받아, 관련 라이브러리를 도입할 계획
- 코드 편집기와 그래프에 커서를 가져가 대면 각 부분에 강조가 되어야 한다는 의견을 받아, 관련 라이브러리를 도입할 계획
- 구축을 한 후 하단에 스크롤 바를 만들어 상태를 드래그 하며 시각적으로 디버깅을 할 수 있도록 할 것

제 3 장

결과보고서

3.1 결과보고서

3.2 결과보고서 발표자료



국민대학교
소프트웨어융합대학
소프트웨어학부

캡스톤디자인 I

종합설계프로젝트

프로젝트 명	TraceInspector - 추적 가능한 시각화 기반 코드 정적분석기
팀 명	팀 2
문서 제목	결과보고서

Version	1.0
Date	2026-06-16

팀원	김신영 (조장)
	강민성



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

목 차

1 개요	2
1.1 프로젝트 개요	2
1.2 추진 배경 및 필요성	2
2 개발 내용 및 결과물	4
2.1 목표	4
2.2 연구/개발 내용 및 결과물	5
2.2.1 연구/개발 내용.....	5
2.2.1.1 Tracelnspector 분석기	5
배경: 정적 프로그램 분석과 결정가능성(decidability)	6
배경: 프로그램 의미론	7
배경: 요약해석(abstract interpretation)	8
Imp 중간 표현 언어	13
요약 도메인.....	15
제어 흐름 그래프(control-flow graph).....	17
요약실행 의미론	18
오류 상태 판별	21
2.2.1.2 Tracelnspector 프론트엔드	21
UI 요약설명	21
작동방식 상세설명	22
사용요소 상세설명	24
2.2.2 시스템 기능 요구사항.....	25
2.2.3 시스템 비기능(품질) 요구사항	26
2.2.4 시스템 구조 및 설계도.....	29
2.2.5 활용/개발된 기술	29
2.2.6 현실적 제한 요소 및 그 해결 방안	30
2.2.7 결과물 목록	31
2.3 기대효과 및 활용방안.....	34
3 자기평가	34
4 참고 문헌	35
5 부록	37
5.1 사용자 메뉴얼	37
5.2 운영자 메뉴얼	38
5.3 배포 가이드.....	38
5.4 Imp 형식 의미론.....	38
5.4.1 요약 도메인 Go 인터페이스	40

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

1 개요

1.1 프로젝트 개요

TraceInspector는 요약해석(abstract interpretation)에 기반한 정적 분석기이다.

1.2 추진 배경 및 필요성

올바르고 안전한 코드를 작성하는 것은 모든 프로그래머의 목표이다. 그러나 현대의 프로그래밍 환경은 고도의 추상화와 복잡한 소프트웨어 구성 요소들로 이루어져 있어, 처음부터 완전히 안전한 코드를 작성하는 것은 사실상 불가능에 가깝다. 대신 개발자는 코드를 작성하고, 오류를 탐지하며, 디버깅을 통해 문제를 수정하는 과정을 반복함으로써 점진적으로 프로그램의 안전성과 신뢰성을 향상시킨다.

이 과정에서 오류를 효과적으로 탐지하는 것은 매우 중요하다. 오류가 탐지되어야만 디버깅 과정이 시작될 수 있으며, 프로그램에 존재하는 문제의 원인과 위치를 파악할 수 있기 때문이다. 따라서 소프트웨어 개발 과정에서는 단위 테스트(unit testing), 통합 테스트(integration testing), 코드 리뷰(code review), 정적 분석(static analysis) 등 다양한 오류 탐지 기법들이 적극적으로 활용된다.

정적 프로그램 분석(static program analysis)이란 프로그램을 직접 실행하지 않고 소스 코드를 분석하여 실제 동작 특성을 추론하는 기법들을 의미한다.

```
infer@facebook:~/infer/examples$ infer -- javac Infer.java
Starting analysis (Infer version v0.5.0)
Computing dependencies... 100%
Analyzing 1 cluster. 100%
Analyzed 2 procedures in 1 file
No issues found
infer@facebook:~/infer/examples$ vim Infer.java
infer@facebook:~/infer/examples$ infer -- javac Infer.java
Starting analysis (Infer version v0.5.0)
Computing dependencies... 100%
Analyzing 1 cluster. 100%
Analyzed 3 procedures in 1 file

Found 1 issue
Infer.java:12: error: NULL_DEREFERENCE
object s last assigned on line 11 could be null and is dereferenced at line 12
10.     int mayCauseNPE() {
11.         String s = mayReturnNull(0);
12. >         return s.length();
13.     }
14.
```

그림 1: Infer 분석기를 이용해 Java 코드의 NullPointerException을 탐지하는 모습

IDE에서 코드를 저장할 때마다 표시되는 unused variable 등의 경보, 컴파일러에서 보다 효율적인 코드 생성을 위해 수행하는 dataflow analysis, CI/CD 절차에서 커밋 후 수행되는 code linter 등 개발의 전 과정에서 프로그래머가 알게 모르게 활용되고 있다.

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

Facebook Infer[2], clang static analyzer[10] 등 실제 프로그래머가 활용하여 오류를 탐지하고자 하는 목적으로 작성된 정적 분석기와, LLVM[11] 등의 컴파일러 내부에서 중간절차로 활용되는 경우와 같이 사용 환경과 분석 목표는 서로 다르지만, 공통적으로 다음과 같은 목표를 가진다:

- 정확성: 의미 있는 오류 정보를 정확하게 제공해야 한다.
- 신속성: 분석 과정이 신속해야 하며, 지나치게 많은 시공간 자원을 소모해서는 안된다.
- 편리성 및 자동화: 최소한의 사용자 개입 및 추가적인 정보 없이 독자적으로 동작해야 한다.

즉 대량의 코드에 대해 빠르고 정확하게 분석하는 것을 목표로 한다.

하지만 우리가 정적 분석기를 활용하면서, 한번쯤은 다음과 같은 질문들을 해볼 수 있다:

- 정적 분석기는 어떤 과정을 통해 코드 속의 오류를 탐지할 수 있을까?
- 이러한 정적 분석기의 결과를 우리가 신뢰할 수 있을까?
- 왜 "정적"일까?

이러한 질문들은 개발자에게 익숙한 일상적 경험으로 인해 간과되기 쉽다. 하지만 단순히 코드를 입력으로 넣었을 때 오류 정보를 알려주는 "마법의 상자"로 생각하지 않고, 그 내부의 추론 과정과 분석 원리를 이해할 수 있다면, 분석 결과를 보다 효과적으로 해석하고 활용할 수 있을 것이다.

TraceInspector는 위와 같은 궁금증을 해소하고자 하는 취지에서 개발되었다. 정적 프로그램 분석 기법 중 요약해석(abstract interpretation) 이론에 대해 소개하고, 단순히 어떤 오류가 존재하는지에 대한 정보를 제공하는 것을 넘어 "어떻게" 해당 이론을 통해 탐지되었는지 시각화하여 보여주하고자 한다. 이를 통해 프로그래머들에게 정적 분석 이론에 대한 이해와 친숙도를 높이고, 단순히 코드를 작성하는 행위를 넘어 논증의 대상으로 바라보는 관점을 소개하고자 한다.

알고리즘 혹은 자료구조 강의를 생각해보자. 이를테면 그래프 탐색 알고리즘을 처음 접했을 때, 알고리즘의 수학적 내용과 의사 코드만 보아서는 온전히 해당 개념을 직관적으로 이해하기 어렵다. 대신 실제 그래프 위에서 동작하는 예시를 통해 비로소 직관적인 이해가 가능해지는 경우가 많다. 특히나 알고리즘과 같이 추상화의 정도가 높은 주제의 경우 더 더욱 그럴 것이다. 마찬가지로 TraceInspector 역시 정적 분석 알고리즘을 시각화하여 동작 원리에 대해 와닿을 수 있는 설명을 제시하는 것이 목표이다.



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

2 개발 내용 및 결과물

2.1 목표

정적 분석기는 정확성과 신뢰성이 중요하다. 실제로 오류를 탐지한 경우 그 정보를 바탕으로 프로그래머는 코드를 검토하고 개발의 방향에 영향을 주는 만큼 중요한 소프트웨어이다. 따라서 점점 프로그래밍 언어와 코드베이스가 복잡해지면서 정적 코드 분석기의 구현에 대한 신뢰성과 안전성을 검증하는것 역시 못지않게 중요하다고 볼 수 있다.

Frama-C[9], ASTRÉE[5]와 같은 정적 분석기들은 광범위하게 활용 가능한 실용적인 정적 분석기임과 동시에 탄탄한 이론적 바탕을 기반으로 하여 신뢰성을 보장하고자 한다. 심지어 Verasco[7] 프로젝트는 Rocq(구 Coq)[19]정리 증명 언어로 정형 검증된 정적 분석기이며, 이에 따라 분석기의 논리적 건정성이 증명된 안전한 분석기이다.

이처럼 프로그램을 "잘" 분석하는 분석기를 구현하기 위해선 신뢰할 수 있는 탄탄한 이론적 바탕이 필요함을 알 수 있게 되었다.

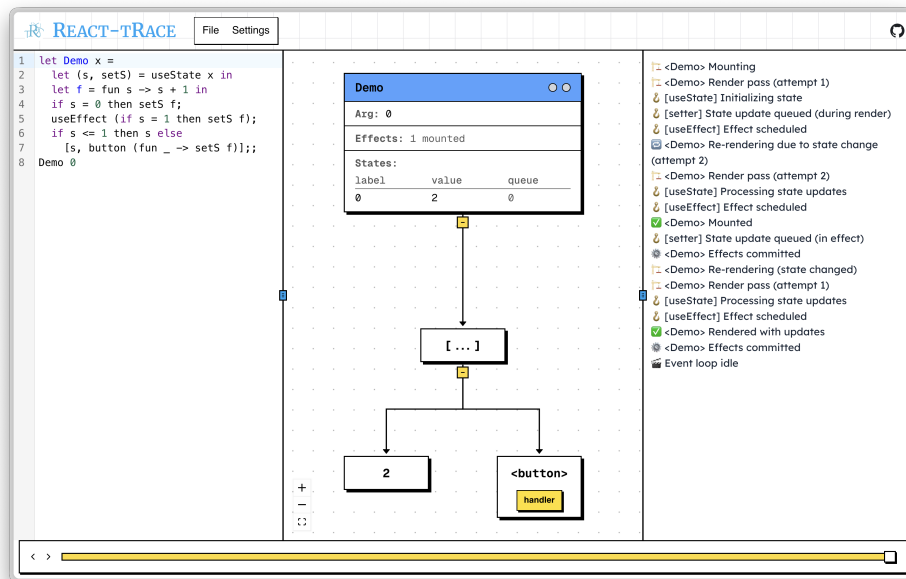


그림 2: react-tRace의 시각화 인터페이스

React-tRace[12]는 React 웹 프레임워크의 Hook가 어떤 과정을 따라 컴포넌트를 변화시키는지 단계적인 형식 의미론을 정의한 연구이다. *React-tRace*의 흥미로운 부분은 기존 형식 의미론 연구와는 다르게 웹 인터페이스(그림 2)를 통해 사용자가 직접 해당 의미론에 따라 웹페이지 컴포넌트의 상태 변화를 직접 관찰할 수 있게 해준 점이다. 따라서 단순히 수식으로 정의한 의미론을 읽는 것을 넘어 각 변화 단계가 어떤 수식에 의해 변했는지 구체적으로 추론하고 이해할 수 있다는 점이 직관적으로 와닿고, 복잡한 알고리즘을 이해하기에 탁월한 방법임을 파악하게 되었다.



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

TraceInspector는 해당 연구로부터 영감을 받아 단순히 오류를 탐지해서 알려주는 정적 분석기를 넘어, 분석 과정을 시각적으로 나타내어 사용자가 어떤 과정을 통해 오류가 탐지되었는지 해석 가능하게끔 구현된 정적 분석기이다. 즉 분석기의 동작 흐름을 사용자가 직접 관찰하며 올바른 추론을 분석기가 수행했는지, 그리고 어떻게 정적 분석이 이루어졌는지를 전달하고자 본 프로젝트를 구상하였다. 다양한 오류를 신속하게 분석하는게 아닌 어떤 과정으로 정적 분석이 프로그램을 처리하고 오류를 탐지하는지에 대한 정보 전달 및 시각화를 목표로 하며, 이에 따른 프로그래밍 언어론과 정적 분석 이론에 대한 교육적 자료로 활용되기를 바란다.

앞서 설명한 목표와 같이, *React-tRace*의 형식 의미론처럼 신뢰 가능하고 이론적으로 건전한 분석기를 구현하는 것을 목표로 하였다. 이에 따라 기존 정적 분석기보다는 높은 수준의 신뢰성을 추구할 수 있도록 설계하였고 요약해석 이론을 도입하여 엄밀한 명세와 정의를 바탕으로 정적 분석기를 구현하였다.

2.2 연구/개발 내용 및 결과물

2.2.1 연구/개발 내용

TraceInspector는 요약해석 이론을 구현하여 소스 코드를 분석하는 "분석기"와 분석과 관련된 정보를 시각화하여 나타내는 "인터페이스"로 구성되어 있다.

다음은 각 요소별 설명이다.

2.2.1.1 TraceInspector 분석기

TraceInspector 분석기는 Go 언어 소스 코드를 입력으로 받아, 다음의 정보를 출력하는 단일 실행 파일(executable)이다:

- 제어 흐름 그래프(control-flow graph): 입력 소스 코드에 대응되는 제어 흐름 그래프
- 오류 정보: 입력 소스에서 발생할 수 있는 오류의 유형과 코드상 위치
- 분석 단계별 상태: 분석을 수행하며 단계적으로 계산된 중간 계산 결과

명령줄 인자를 통해 .go 파일을 입력받으며, 위의 정보는 모두 표준화된 JSON 형식으로 출력된다. 이에 따라 향후 TraceInspector 인터페이스 뿐만 아니라 IDE와의 통합 등도 가능하다.

정적 분석은 "프로그램의 실제 실행 동작을 코드 분석을 통해 추론하는 과정"이다. 그렇다면 실제 실행 동작이 무엇인지, 코드 분석은 어떻게 할 것인지, 그리고 무엇을 추론할지에 대해서도 정의해야 한다. 각 요소들이 어떻게 정의되는지 보다 엄밀한 소개를 하고자 한다.

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

배경: 정적 프로그램 분석과 결정가능성(decidability)

상술했듯이 정적 프로그램 분석은 실행하지 않고 동작 특성을 추론하는 기법이다. 동작 특성의 경우 다음과 같은 예시들이 존재한다:

- 0으로의 나눗셈을 수행하게끔 하는 입력이 존재하는가?
- 배열 인덱싱은 항상 올바른 범위 내에서 수행되는가?
- null 포인터를 참조하지 않는가?
- 로그인하지 않은 상태에서 사용자 정보에 접근할 수 있는가?
- 모든 입력에 대해 프로그램이 유한한 시간 내에 종료하는가?

이처럼 실행했을 때의 행동과 관련되어 있고, 참 또는 거짓으로 판별되는 명제들을 의미론적 성질(semantic property)로 칭하게 된다. "의미론적" 성질인 이유는 단순히 프로그램의 겉모양인 구문(syntax)만 분석해서 판별할 수 없기 때문이다.

코드 1은 콜라츠 수열(Collatz sequence)의 n 번항을 계산하는 함수이며 다음과 같은 정의를 가진다:

$$C(n) = \begin{cases} \frac{n}{2} & n \text{ 은 짝수인 경우} \\ 3n + 1 & n \text{ 은 홀수인 경우} \end{cases}$$

```
def collatz(n):
    if n == 1:
        return 1
    if n % 2 == 0:
        return collatz(n / 2)
    else:
        return collatz(3 * n + 1)
```

코드 1: Collatz 수열을 계산하는 코드

모든 양수 n 에 대해서 위 함수가 종료할까? 1937년도에 수학자 콜라츠는 "그렇다"고 추측했지만, 아직까지도 그가 맞았다는 증명이 존재하지 않는다. 이에 따라 특정 입력값에 대해서는 실행을 통해 확인할 수 있지만, 모든 양수 입력에 대해 종료함을 보장하는 일반적인 방법은 현재까지 알려져 있지 않다.

비슷하게 튜링의 유명한 정지 문제(halting problem)[20]가 있다: 튜링의 정지 문제는 "임의의 프로그램이 주어졌을 때, 해당 프로그램이 유한한 시간 내에 정지할지 정확히 판별하는 프로그램은 존재할 수 없다"는 정리이다.



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

콜라츠 추측과 정지 문제와 같이 우리는 답을 정확히 알 수 없는 의미론적 성질이 존재함을 알게 되었다. 그렇다면 비자명한 의미론적 성질 중 자동으로 완벽하게 판별 가능한 것이 단 하나라도 존재할까? 놀랍게도 Rice 정리에 의하면 **존재하지 않는다**:

정리 (Rice[16], 1953). L 이 튜링 완성 언어이고, P 가 L 의 프로그램들에 대한 비자명한 의미론적 성질일 때, L 의 모든 프로그램 p 에 대해

$$A(p) = \begin{cases} 참 & p \text{ 가 } P \text{ 를 만족하는 경우} \\ 거짓 & p \text{ 가 } P \text{ 를 만족하지 않는 경우} \end{cases}$$

를 만족하는 판별 알고리즘 A 는 존재하지 않는다.

이를 풀어 말하자면 어떤 튜링 완성 언어로 작성된 임의의 프로그램 p 가 성질 P 를 만족하는지 완벽하게 판별 가능한 알고리즘은 존재할 수 없다는 의미이다. 여기서 P 를 "프로그램이 언젠가는 종료한다"로 한정하면 정지 문제가 된다.

논리학과 계산과학에서는 이처럼 어떤 명제에 대해 참 또는 거짓을 판별할 수 있는 알고리즘의 존재 여부를 결정가능성(decidability)[14]이라고 칭한다. 이러한 관점에서 정적 프로그램 분석은 의미론적 성질을 판별하는 문제이기에 결정 불가능(undecidable)하며, 이에 따라 "모든 프로그램"에 대해 "자동으로", "완벽한" 정확성을 가지는 정적 분석기는 구현이 불가능하다.

결국 "모든 프로그램에 대해 동작", "사용자의 개입 없는 알고리즘", "완벽한 정확성(분석 결과가 실제 오류 존재 여부와 동치)" 중 최소한 하나의 조건은 포기해야 분석기 구현이 가능해진다.

배경: 프로그램 의미론

프로그램이 실행되었을 때 동작하는 과정을 어떻게 표현할까? 단순히 생각하면 메모리의 특정한 위치에 특정한 값을 계산한 후 저장한다고 정의할 수 있다. 형식 프로그램 의미론(formal program semantics)은 이러한 프로그램의 동작 과정에 대한 엄밀한 수학적 모형을 제시한다. 본 문서에서는 형식 의미론 중 operational semantics에 대해 소개하고자 한다. Tracelnspector의 요약해석 구현이 operational semantics 위에서 정의되었기 때문이다.

Operational semantics는 프로그램 실행 과정을 프로그램 상태간의 전이로 나타내는 기법이다. 그림 3는 어떤 시작 메모리 상태 S 로부터 프로그램이 실행을 시작해서 $a_1, a_2, \dots, a_n$ 중 하나의 상태로 전이되고, 이후에 후속 상태들로 계속해서 전이하는 구조로 나타낸 예시이다. 마치 오토마타에서 시작 상태에서부터 주어진 규칙과 입력에 의해 다음 상태로

¹non-trivial: 일부 프로그램에 대해서는 참이고, 일부 프로그램에 대해서는 거짓인 성질



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

전이되는 것과 같이, 프로그램 의미론 역시 주어진 상태와 입력에서 언어의 문법적 요소별로 정해진 규칙에 따라 다음 상태로 전이되는 체계로 나타낸 형식이다.

요약해석 맥락에서는 이처럼 실제 프로그램 실행을 묘사하는 의미론을 "구체적 의미론 (concrete semantics)"이라 칭한다. TraceInspector에서 정의한 구체적 의미론 전체는 ??에 수록하였다.

이러한 구체적 의미론 위에서 주어진 입력과 초기 상태에 대해 실행 시 거치는 상태들의 연속을 실행 흔적(trace)라 칭한다. $S \hookrightarrow a_1 \hookrightarrow b_1 \hookrightarrow c_1$ 흔적을 예시로 살펴보면, 시작 상태 S 로부터 a_1, b_1 을 거쳐 최종 상태 c_1 에서 종료된 하나의 실행 흔적이다.

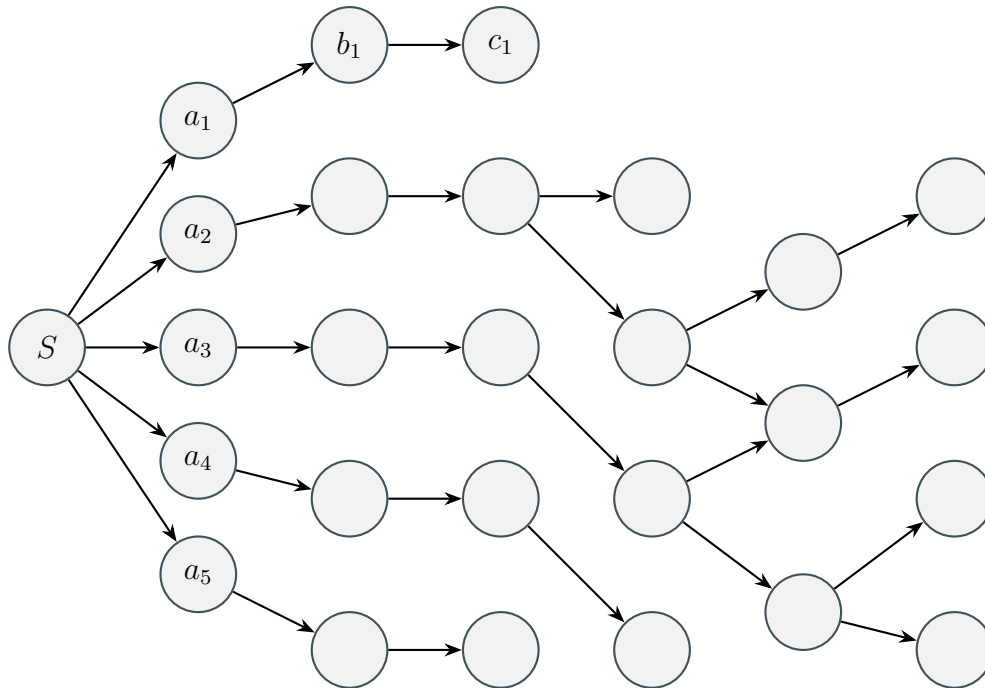


그림 3: 프로그램 상태 간 전이를 통해 의미를 나타내는 operational semantics

오류 상태란 어떤 명제 P 를 만족하는(혹은 만족하지 않는) 상태들의 집합이라 볼 수 있다. 그림 4에선 오류 상태들을 $e_1, \dots, e_7$ 로 정의하였다. 이에 따라 프로그램의 초기 상태에서부터 시작하여 e_n 에 도달 가능한 경로 $S \hookrightarrow a_n \hookrightarrow \dots \hookrightarrow e_n$ 가 존재하는 경우 오류가 발생할 수 있는 프로그램으로 정의할 수 있다.

배경: 요약해석(abstract interpretation)

이제 프로그램이란 메모리 상태간의 전이이고, 전이 방법은 해당 프로그래밍 언어의 형식 의미론에 의해 정해지며, 오류란 특정한 조건을 만족하는 메모리 상태이며, 시작 상태에서부터 도달 가능한 오류 상태가 존재하는 경우 오류가 존재하는 프로그램이라는 것을 정의하였다.

프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

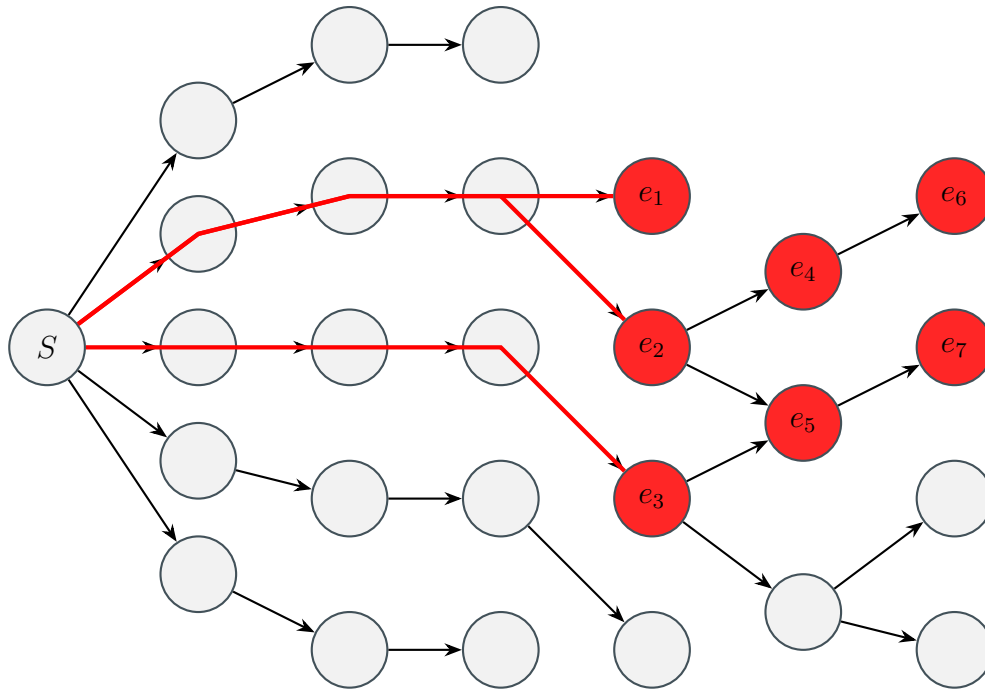


그림 4: 오류 상태 및 흔적

요약해석 이론은 논리적으로 건전한(sound) 분석을 목표로 한다. 신뢰할 수 있는 분석기를 만든다는 것은 다양한 해석이 가능하겠지만 그중 "오류가 없다고 분석된다면 실제로 실행 가능한 모든 경로에 대 오류 흔적은 존재하지 않는다" 고 정의할 수 있으며, 이 문장이 논리적 건정성을 한 문장으로 표현한 것이다.

입력 프로그램 p 에 대해, $analysis(p)$ 는 p 에 대한 분석을 수행하며, 그 결과로 "오류가 존재한다"와 "오류가 존재하지 않는다"를 반환한다. $error(p)$ 는 p 를 실행했을 때 오류가 발생 가능한지 판별하는 오라클(oracle) 함수이다. 여기서 오라클이란 실제로 구현 불가능하지만 완벽하게 사실을 판별할 수 있는 가상의 함수이다. $analysis(p)$ 가 논리적으로 건전하기 위해선

$$analysis(p) = \text{"오류가 존재하지 않는다"} \Rightarrow error(p) = \text{"오류가 존재하지 않는다"} \quad (1)$$

를 만족해야 한다.² 이 조건은 오류가 존재하지 않는 분석 결과에 대해서는 보증을 하지만, 반면 분석 결과가 "오류가 존재한다"인 경우 어떠한 보증도 하지 않게 된다. 실제로 실행했을 때 오류가 발생할수도 있지만 그렇지 않고 존재하지 않는 오류에 대한 허위 정보(false positive)를 낼 수도 있다는 의미이다. 앞서 설명한 Rice 정리의 직접적인 영향 때문이다.

요약해석 이론은 실제 프로그램 상태에 대한 "보수적인" 근사치를 논리적으로 건전하게 계산하는 방법이다. 앞서 활용한 프로그램 흔적 예시를 요약해석의 관점에서 생각해보자

² 더 일반적이고 엄밀한 정의는 임의의 의미론적 명제 P 에 대해 $analysis(p, P) = true \Rightarrow P$ p 이다.

프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

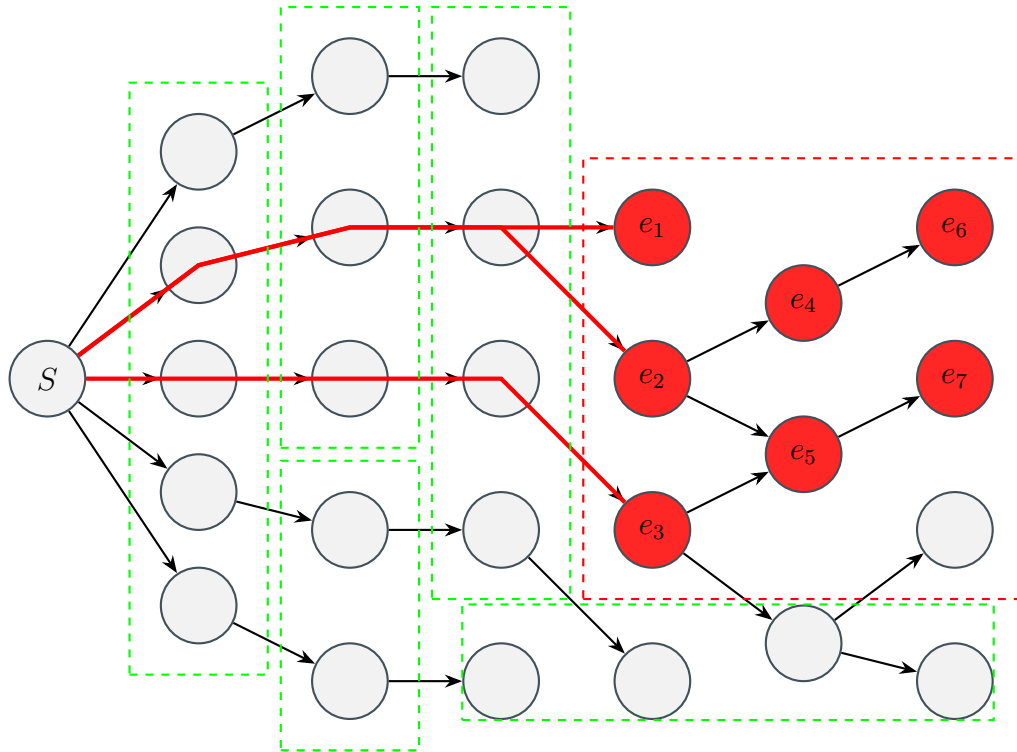


그림 5: 오류 상태의 요약

면, 그림 5와 같이 모든 상태들에 대한 구체적인 전이를 계산하지 않고, 초록색 상자처럼 여러개의 구체적인 상태들을 요약해서 나타낼 수 있는 "요약 상태"로 압축하여 나타내는 것이다. 오류 상태들에 대해서도 마찬가지로 구체적인 오류 상태들을 포함한 근사치(빨간색 상자)로 오류 상태를 정의하게 된다. 보수적인 근사이기 때문에 요약 오류 상태에서는 실제로는 오류 상태가 아니지만 포함된 일반 상태도 존재하게 된다. 하지만 실제 오류 상태들은 빠짐없이 요약 상태에서도 오류로 표현되어야 한다.

```
x = input()[x : n]
if x < 0:
    y = -x [x : n, y : -n]
elif x > 0:
    y = x [x : n, y : n]
else:
    y = 0 [x : n, y : n]
```

(a) 구체적 정수값

```
x = input()[x : T]
if x < 0:
    y = -x [x : -, y : +]
elif x > 0:
    y = x [x : +, y : +]
else:
    y = 0 [x : 0, y : 0]
```

(b) 부호 도메인으로 요약

그림 6: 정수값과 부호 도메인

그림 6는 간단한 정수값을 계산하는 프로그램에 대해 구체적인 값들을 계산하는 경우와 각 변수의 부호만을 요약해서 계산하는 경우를 나타내었다. 구체적인 값의 경우 x가 임의의



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

입력 $n \in \mathbb{Z}$ 을 받은 이후, 코드에 따라 변수 값을 직접 계산하게 된다. 반면 부호로 요약한 경우 잠재적으로 가능한 입력은 모든 정수이기에, $-, 0, +$ 중 어떤 값도 가능한 $\top$ 기호를 통해 나타나게 되고, 이후 조건에 따라 양수, 음수, 0 으로 나뉘서 표현하게 된다. 부호 정보만을 가지고 있기에 실제로 정수의 값을 알 순 없지만, 실제 값을 알지 못하더라도 각 분기에 들어갔을 때 가질 수 있는 값들에 대한 성질을 추론할 수 있게 된다.

조금 더 엄밀하게 생각해보면 각 코드의 지점마다 변수가 가질 수 있는 값들은 어떤 조건을 만족하는 집합으로 나타낼 수 있다. $x = \text{input}()$ 의 경우 정수집합 $\mathbb{Z}$ 가 가능하며, 양수의 경우 양수들의 집합 $\mathbb{Z}_+ := \{n \mid n > 0\}$, 음수는 $\mathbb{Z}_- := \{n \mid n < 0\}$, 그리고 한원소 집합 $\{0\}$ 로 표현된다. 따라서 정수값들을 다루는 의미에서 프로그램 상태는 정수집합 $\mathbb{Z}$ 의 멍집합 $\mathcal{P}(\mathbb{Z})$ 가 된다.

부호들의 경우 보다 단순하다. "모든 부호"를 나타내는 $\top$ 와, $+, 0, -$ 로 구성되어 있다. 추가로 위 코드에서는 쓰이지 않았지만, "모든 부호"의 쌍대에 해당하는 "유효하지 않은 값들의 부호"를 나타내는 $\perp$ 도 추가된다. 즉 부호로 요약해서 계산하는 의미론에서는 프로그램 상태가 부호 요약도메인 $\text{sign} := \{\top, +, 0, -, \perp\}$ 의 원소들로 표현된다.

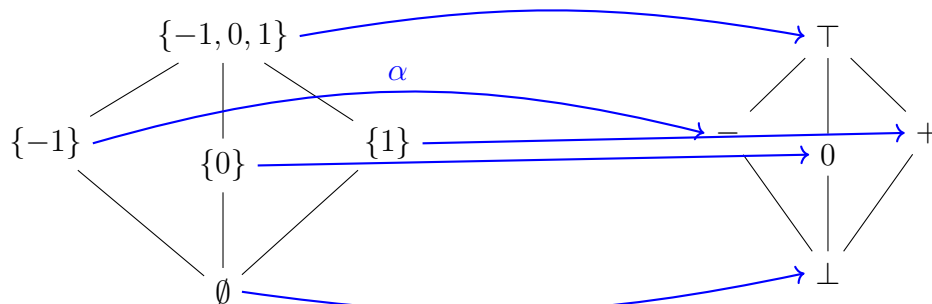


그림 7: 정수 집합과 부호로의 대응

따라서 부호를 계산하는 것이란 주어진 코드 지점에서 구체적인 정수값들의 집합 $s \subset \mathbb{Z}$ 에서 sign 으로 대응시키는 함수 $\alpha : \mathcal{P}(\mathbb{Z}) \rightarrow \text{sign}$ 으로 볼 수 있다. $x = \text{input}()$ 를 수행한 이후 x 가 가능한 값들의 집합은 $\mathbb{Z}$ 이고 이에 따른 $\alpha(\mathbb{Z}) = \top$ 가 되고, 마찬가지로 x 가 양수인 집합은 $\mathbb{Z}_+$, $\alpha(\mathbb{Z}_+) = +$ 가 된다.

$x := 4$ 와 같은 코드가 실행된 이후 가능한 상태의 집합은 한원소 집합 $\{4\}$ 이다. $\alpha(\{4\}) = +$ 가 되는데, 이는 $\alpha(\mathbb{Z}_+)$ 와 동일한 결과를 가지게 된다. 양수들의 집합보다 구체적인 정보를 가지고 있었지만 부호로 요약되며 해당 정보를 잃게 되는 것이다. 그림 6는 정수 집합을 부호 요약도메인으로 대응시키는 예시를 보여준다.

구체적인 값들의 집합에서 요약도메인의 원소로 대응시키는 α 함수가 있듯이, 역으로 요약도메인의 원소에서 구체적인 값들의 집합으로 대응시키는 $\gamma : \text{sign} \rightarrow \mathcal{P}(\mathbb{Z})$ 를 정의할 수 있다. $\gamma(\top) = \mathbb{Z}, \gamma(+)=\mathbb{Z}_+, \dots$ 로 간단하게 정의된다.

두 함수 α, γ 에 대해서 다음과 같은 성질에 대해 생각해볼 수 있다:



- 임의의 정수집합 $s \subseteq \mathbb{Z}$ 에 대해, $\gamma(\alpha(s))$ 의 결과
- 임의의 부호도메인의 원소 $e \in \text{sign}$ 에 대해, $\alpha(\gamma(e))$ 의 결과

첫번째 경우는 $\alpha(s)$ 를 수행하며 구체적인 정보가 소실된다. 이에 따라서 $\{4\} \subset \mathbb{Z}_+$ 와 같이 보다 구체적인 집합을 넣더라도 같은 도메인 값 $\alpha(s)$ 으로 요약된다. $\gamma(e)$ 는 해당 부호를 만족하는 원소들을 전부 포함시키기에, 모든 $s \subseteq \mathbb{Z}$ 에 대해 $s \subseteq \gamma(\alpha(s))$ 임을 알 수 있다.[4]

$\gamma(e)$ 의 결과는

- $\gamma(\top) = \mathbb{Z}$
- $\gamma(\perp) = \emptyset$
- $\gamma(+) = \mathbb{Z}_+$
- $\gamma(0_{\text{sign}}) = \{0\}$
- $\gamma(-) = \mathbb{Z}_-$

로 열거할 수 있다. 따라서 모든 $e \in \text{sign}$ 에 대해 $e = \alpha(\gamma(e))$ 임을 알 수 있다.

α, γ 두 함수에 대한 이 성질들은 "보수적인" 요약이 무엇인지를 정확하게 나타내는 역할을 한다. 구체적인 값들의 집합을 요약하여 표현한 결과는 항상 원래 값을 포함하여야 함을 정의한 것이고, 이 조건을 통해 요약 상태에서 추론을 하더라도 구체적인 상태를 빠짐없이 반영할 수 있음을 보장하게 된다.

지금까지 설명은 정수들의 집합 $\mathcal{P}(\mathbb{Z})$ 에 대한 순서관계 $\subseteq$ 가 정의된 상태에서 진행되었는데 마찬가지로 요약도메인 sign 에 대해서도 순서관계 $\sqsubseteq$ 를 정의할 수 있다: $\perp$ 는 어떠한 값도 나타내지 않기에 $\forall e. \perp \sqsubseteq e$ 가 되며, 마찬가지로 $\top$ 는 모든 정수값을 나타내기에 $\forall e. e \sqsubseteq \top$ 가 성립된다. $+, 0, -$ 는 구체적인 상태로도 서로소 집합이기에 포함관계로도 순서 정의가 불가능하다. 이는 "부분 순서 관계(partially ordered)"로 정의가 되며, 모든 원소보다 순서 관계상 큰 $\top$ 원소와 작은 원소 $\perp$ 가 존재하기에 "완전 부분 순서 관계(complete partial order)"의 성질을 가지게 된다.

따라서 위에서 정의한 α, γ 의 두 성질은 순서 관계가 정의된 두 집합 $(\mathcal{P}(\mathbb{Z}), \subseteq)$ 와 $(\text{sign}, \sqsubseteq)$ 사이의 "갈루아 연결(galois connection)"을 나타낸다:

정의 (갈루아 연결(Galois Connection)). 프로그램의 구체적 상태들의 집합 C 와 그 위에서의 순서 관계 $\subseteq$ 가 주어지고 요약상태 집합 A 와 그 위에서의 순서 관계 $\sqsubseteq$ 가 주어졌을 때, 두 함수 $\gamma: A \rightarrow C$ 와 $\alpha: C \rightarrow A$ 에 대해

$$\forall c \in C. \forall a \in A. \alpha(c) \sqsubseteq a \iff c \subseteq \gamma(a)$$

를 만족하면 두 집합과 순서관계 사이에 갈루아 연결을 형성한다고 정의하며,

$$(C, \subseteq) \overset{\alpha}{\underset{\gamma}{\rightleftharpoons}} (A, \sqsubseteq)$$

로 나타낸다.[3][17]

이는 단순히 "정수와 부호"와 같이 특정한 값들과 요약 상태뿐만 아니라 일반적으로 적용할 수 있는 성질이며, 따라서 요약해석에 기반한 정적분석을 구현할 때 논리적으로 건전함을 보이기 위해선 프로그램의 구체적 상태와 요약 상태간의 갈루아 연결이 형성됨을 증명해야 한다.

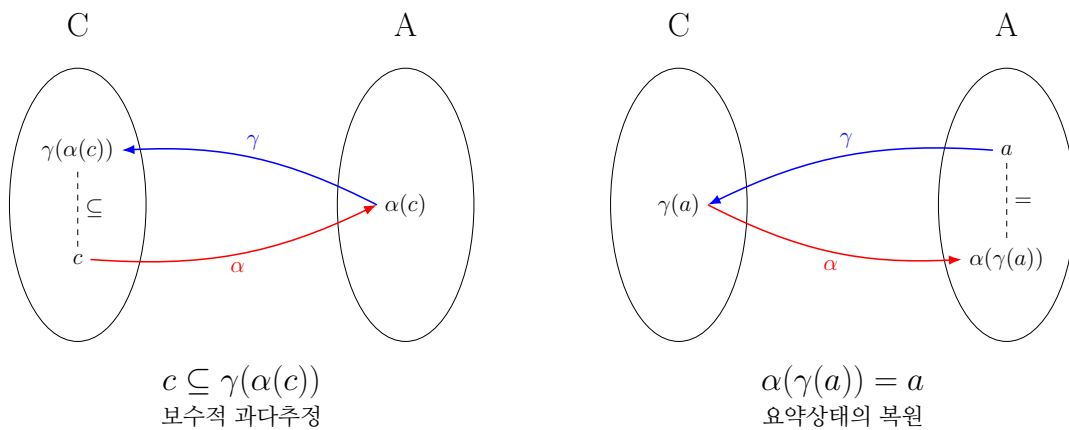


그림 8: 갈루아 연결과 α, γ 의 성질

TraceInspector는 algebra/extended_integers.go에 $\pm\infty$ 를 추가한 확장된 정수체계 $\mathbb{Z}_\infty$ 를 구현하였다. 대소관계 비교 및 산술연산이 구현되어 있으며, 이를 기반으로 domain/interval.go에 확장된 정수체계 기반 구간도메인이 구현되어 있다.

Imp 중간 표현 언어

TraceInspector의 첫 단계는 입력으로 Go 소스 파일을 받아 Go 표준 라이브러리의 go/parser 패키지를 이용해 추상 구문 트리(abstract syntax tree, AST)로 파싱하는 것이다. 이후 Imp[15]라는 간단한 명령형 프로그래밍 언어 겸 중간 표현 형식으로 번역한 후 ImpAST위에서 분석 단계를 수행한다.

Imp는 다음과 같은 기능을 가진 명령형 언어이다:

- int, bool, string 및 다차원 동적 배열 []T
- 산술/논리 연산 및 입출력
- if, while, continue, break 제어 흐름



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

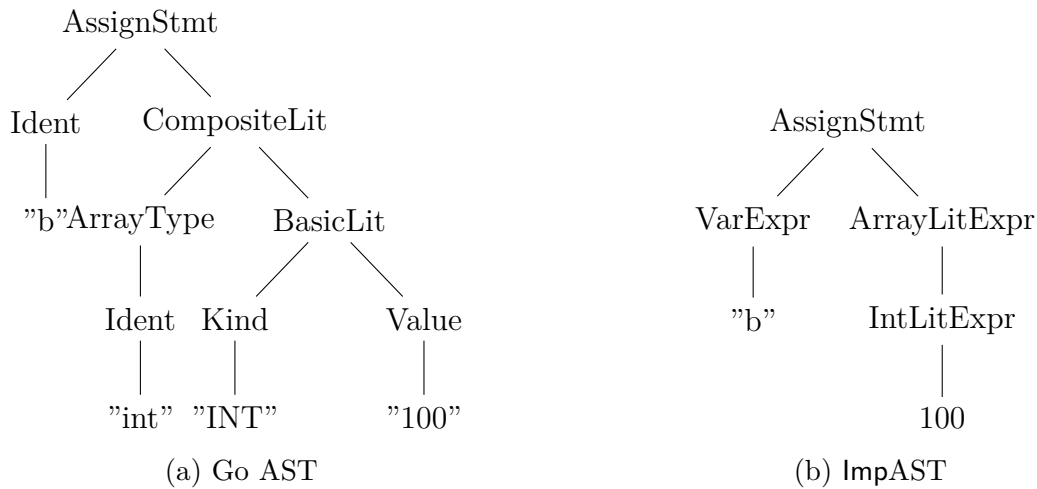


그림 9: Go AST와 ImpAST 비교

- 함수 선언 및 반환

이처럼 구조적으로 간단하지만 충분히 복잡한 수준의 프로그램을 표현할 수 있는 구조이다.

Go AST에 대해 직접적으로 분석을 수행하지 않는 이유는 크게 두가지가 있는데, 첫 번째는 추상 구문 트리의 특성상 프로그램의 제어 흐름을 제외한 구문적 요소들이 트리에 많이 포함되어 있기 때문이다. 그림 9는 b라는 변수에 원소가 100인 정수형 동적 배열을 선언하는 명령 `b := []int{100}`를 Go AST로 파싱한 결과와, 이를 ImpAST로 번역한 결과를 나타낸다. Go AST의 경우 리터럴에 대해서도 종류와 값을 분리하여 표기하고, 배열 리터럴 역시 타입을 구별하여 표기하는 등 범용적이지만 복잡한 형태로 구성되어 있다.

반면 Imp의 경우 리터럴의 타입별 정의와 배열 리터럴 등 문법적 요소를 최대한 배제한 형태로 구현하였고, 리터럴 값을 문자열로 표현한 것이 아니라 타입에 맞는 값을 직접 담게 된다. 이러한 설계는 향후 제어 흐름 그래프 생성 등 프로그램의 형태가 아닌 동작 특성을 파악하고자 하는 경우에 훨씬 간결하고 수월하게 구현을 가능케 해준다.

두번째 이유는 특정 언어의 문법적 형태와는 독립적인 분석 기능을 구현하고자 했기 때문이다. Go 언어의 요소에 대한 요약의미론을 직접 정의할 경우 오직 Go 언어에 대해서만 분석 가능하지만, Imp라는 전형적인 언어를 대상으로 분석기를 구현하고자 할 경우, 향후 Go 외의 타 언어를 Imp로 번역하는 번역기만 구현할 경우 이후 분석기 로직을 온전히 활용할 수 있다.

이처럼 입력 소스를 파싱하고 처리하는 프론트엔드와, 분석을 담당하는 미들엔드를 분리하여 각각 다른 표현상에서 동작하게끔 설계하는 방식은 현대 컴파일러에서 자주 활용되는 매우 전형적인 설계 방식이다. TraceInspector 역시 이러한 설계 패턴을 도입하고, 구현의 편의성을 위해 Imp로의 번역 과정을 최우선적으로 수행한다.

Imp의 문법적 정의는 그림 10과 같으며, Imp의 bigstep operational semantics 일부는 부록 5.4에 수록하였다.



$$\begin{aligned}
 e &::= \mathbb{X} \mid \mathbb{N} \mid \mathbb{B} \mid \mathbb{S} \mid \{e_1, \dots, e_n\} \\
 &\quad | e + e \mid e - e \mid e * e \mid e / e \mid e \bmod e \\
 &\quad | e = e \mid e \neq e \mid e < e \mid e \leq e \mid e > e \mid e \geq e \\
 &\quad | e \wedge e \mid e \vee e \\
 &\quad | \neg e \mid \neg e \\
 &\quad | (e) \mid e[e] \\
 &\quad | f(e_1, \dots, e_n) \\
 &\quad | \text{len}(e) \\
 &\quad | \text{make_array}(e, e) \\
 \\
 s &::= \text{skip} \\
 &\quad | x := e \\
 &\quad | e[e] := e \\
 &\quad | \text{if } e \text{ then } \{s^*\} \text{ else } \{s^*\} \\
 &\quad | \text{while } e \text{ do } \{s^*\} \\
 &\quad | \text{break} \mid \text{continue} \\
 &\quad | e++ \mid e-- \\
 &\quad | f(e_1, \dots, e_n) \\
 &\quad | \text{print}(e_1, \dots, e_n) \\
 &\quad | \text{scanf}(s, e_1, \dots, e_n) \\
 &\quad | \text{return } e \\
 \\
 \tau &::= \text{int} \mid \text{bool} \mid \text{string} \\
 &\quad | \text{array}(\tau)
 \end{aligned}$$

그림 10: Imp의 명령문(statement), 식(expression)들의 문법

요약 도메인

TraceInspector는 정수값을 구간 요약도메인으로, bool값을 bool 요약도메인으로, 그리고 배열을 배열 축약 요약도메인으로 요약하여 실행한다.

구간 요약도메인 $\mathbb{I}$ 란 가능한 정수들의 집합을 $[l, h]$ 의 닫힌 구간으로 나타낸 요약 도메인이다.

구간 요약도메인에 대한 α, γ 정의는 다음과 같다:

- $\alpha(\mathbb{Z}) = [-\infty, \infty] = \top$
- $\alpha(\emptyset) = \perp$
- $\alpha(\{x_0, x_1, \dots, x_n\}) = [\min(x_0, x_1, \dots, x_n), \max(x_0, x_1, \dots, x_n)]$
- $\gamma(\top) = \mathbb{Z}$



- $\gamma(\perp) = \emptyset$
- $\gamma([l, h]) = \{i \in \mathbb{Z} \mid l \leq i \leq h\}$

구간끼리의 산술은 다음과 같다:

$$[l_1, u_1] + [l_2, u_2] = [l_1 + l_2, u_1 + u_2]$$

$$[l_1, u_1] \times [l_2, u_2] = [\min(l_1 u_1, l_1 u_2, l_2 u_1, l_2 u_2), \max(l_1 u_1, l_1 u_2, l_2 u_1, l_2 u_2)]$$

$$-[l, u] = [-u, -l]$$

$$i_1 \sqcup i_2 = \begin{cases} i_2, & \text{if } i_1 = \perp \\ i_1, & \text{if } i_2 = \perp \\ [\min(l_1, l_2), \max(u_1, u_2)] & \text{where } i_1 = [l_1, u_1], i_2 = [l_2, u_2] \end{cases}$$

갈루아 연결을 형성하기 위해 구간도메인 $i_1, i_2 \in \mathbb{I}$ 간의 순서관계 $i_1 \sqsubseteq i_2$ 를 다음과 같이 정의한다:

$$i_1 \sqsubseteq i_2 = \begin{cases} \text{true}, & \text{if } i_1 = \perp \\ i_1 = \perp, & \text{if } i_2 = \perp \\ l_2 \leq l_1 \wedge u_1 \leq u_2, & \text{where } i_1 = [l_1, u_1], i_2 = [l_2, u_2] \end{cases}$$

즉 i_1 이 i_2 의 범위 내에 완전히 포함되어 있는 경우 $i_1 \sqsubseteq i_2$ 가 성립한다.

그림 11은 정수 요약도메인을 격자로 표현한 그림이다³. 이렇게 정의된 정수 구간도메인은 그 격자의 높이가 무한히 높다. 따라서 widening 기법을 적용하여 무한히 증가하는 join을 강제로 종료시켜 주기 위한 widening 연산 ∇ 이 필요하다:

$$[n, p] \nabla [n, q] = \begin{cases} [n, p], & \text{if } p \geq q \\ [n, +\infty], & \text{if } p < q \end{cases}$$

Widening 연산은 일정 횟수 이상 상한(혹은 하한)이 단조 증가(감소)하는 경우 정확도를 희생하여 모든 정수들의 상계(하계)인 $\pm\infty$ 를 치환하여 분석이 종료할 수 있도록 유도하는 역할을 한다.

³Patrick Cousot: Principle of Abstract Interpretation의 33.1장에서 발췌함

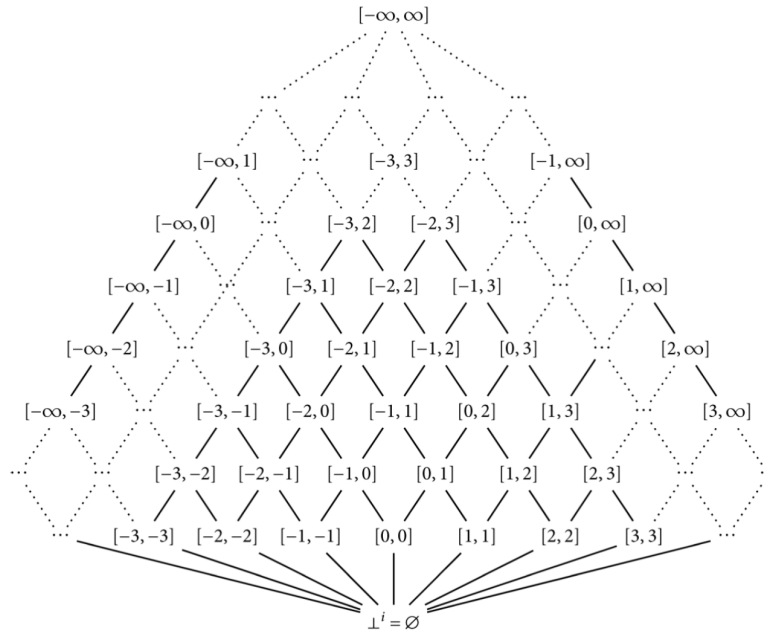


그림 11: 확장된 정수 구간도메인 $\mathbb{I}_{\mathbb{Z}_{\infty}}$

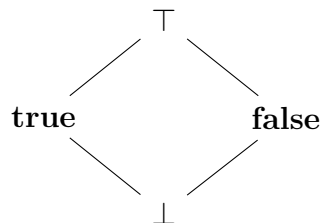


그림 12: Boolean 요약도메인

Boolean 값은 **true**, **false** 만 존재하기에 요약도메인을 그림 12과 같이 정의할 수 있다. $\top$ 의 경우 **true**, **false** 가 모두 가능한 경우이며, $\perp$ 는 유효한 boolean 값 산출이 불가능한 경우이다.

제어 흐름 그래프(control-flow graph)

간략화된 AST가 생성된 이후 TraceInspector는 if, while등의 명령문에 의해 프로그램의 실행 흐름을 구조화하기 위한 제어 흐름 그래프(control-flow graph, CFG)구조[1]로 번역한다. 함수별 AST에서 CFG를 생성하는 알고리즘은 다음과 같이 동작한다:

1. 함수의 진입 명령문으로부터 깊이 우선 탐색을 통해 함수의 종말 명령문부터 노드로 번역하여 ID를 부여한다.
2. 후속 노드의 DFS가 종료되면서 현재 노드와 이어줄 간선을 생성한다. 만약 현재 노드

 국민대학교 소프트웨어학부 다학제간캡스톤디자인I	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

```

fun main() none {
    a = 5
    arr = make_array(a, 1337)
    i = 0
    while i < 10 {
        arr[i] = i
        i = i + 1
    }
    print(arr)
}

```

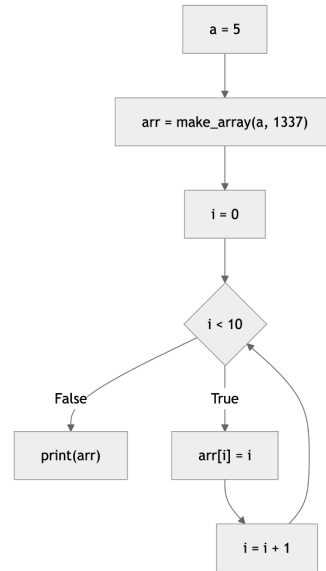
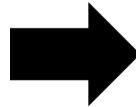


그림 13: 입력 Imp코드와 CFG로의 표현

가 조건문, 반복문과 같이 분기를 포함하는 경우 각 참, 거짓 명령문별 노드와 간선을 생성해준다.

3. `break`, `continue`는 현재 동작하고 있는 가장 깊은 반복문에 대해 동작한다. 이를 처리하기 위해서 현재 처리중인 반복문을 스택 구조로 저장하여 중첩된 반복문에 대한 정보를 저장한다.
4. 이에 따라 `break`, `continue` 및 루프 탈출은 반복문 스택의 top에 대해 동작하며, 반복문을 처리 완료했다면 스택에서 반복문 정보를 제거한다.

그림 13는 예시 Imp코드에 대한 CFG 표현 결과를 나타낸다. 각각의 명령문들이 단일 노드로 표현되며, `while i < 10 {...}` 반복문을 `i < 10` 조건을 하나의 단일 노드로 표현하고 판정 결과에 따라 후속 명령문들을 실행하는 형태로 나타낸 것을 확인할 수 있다.

이러한 CFG구조는 프로그램의 실행 흐름을 직접적으로 노출시키며, `if`, `while`, `break`, `continue` 등의 문법적인 경우를 전부 열거할 필요 없이 조건 노드와 참, 거짓 후속 노드에 대해서만 논할 수 있도록 분석을 간략하게 해준다. 따라서 이후에 TraceInspector분석기는 제어 흐름 그래프 위에서 직접적으로 동작하는 형태로 설계할 수 있으며, Imp언어의 문법적 디테일로부터 보다 자유롭게 추상화된 형태로 동작이 가능하다.

요약실행 의미론



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

이제 입력 프로그램을 CFG 형태로 표현하여 분석이 수월한 형태의 자료구조로 번역하였다. 또한 Imp의 구체적인 값 타입들을 전부 각각의 요약도메인으로 대응시키는 α 함수도 정의하였다.

Imp의 구체적인 실행의미론에선 프로그램 상태 $\sigma : \mathbb{V} \rightarrow v$ 는 각 변수를 값으로 대응시켰다(부록 참조). 또한 명령문 S 를 프로그램 상태 σ 에서 실행했을 때는 결과 상태 σ' 와 제어 흐름 결과 r 를 반환하며 $\langle S, \sigma \rangle \Downarrow \langle \sigma', r \rangle$ 로 표기했다.

Imp의 실제 실행이 구체적인 값 $\mathbb{V}$ 에 대해 계산을 하는 의미론에 바탕을 두었다면, 분석을 위해선 값들의 요약도메인 A 위에서 실행되는 요약실행 의미론을 정의해야 한다.

요약실행 의미론에서 요약된 값은 $\text{AbstractValue} : \tau \rightarrow v_r^\#$ 로 표현된다. 즉 타입 τ 를 해당 타입의 요약도메인 상에서의 값 $v_r^\#$ 로 대응하는 구조가 일차적으로 정의된다.

AbstractValue 는 각 변수마다 존재하기에, 주어진 프로그램 상태에서 각 변수 $v \in \mathbb{V}$ 를 각각의 AbstractValue 로 대응하는 $\text{AbstractVarMemMap} : \mathbb{V} \rightarrow \text{AbstractValue}$ 로 명령문을 실행하는 시점에서의 변수들의 요약 상태를 나타낸다.

이후 CFG 노드별 ID $l \in \mathbb{L}$ 를 AbstractVarMemMap 로 대응시키는 함수 단위의 대응 $\text{AbstractNodeMemMap} : \mathbb{L} \rightarrow \text{AbstractVarMemMap}$ 가 정의된다. 여기서 AbstractValue 는 노드 l 에서 "해당 명령을 실행하기 전의 프로그램 상태", 즉 precondition 상태를 나타낸다.

최종적으로 프로그램 전체의 요약 메모리 상태는 입력 프로그램의 함수들 $f \in F$ 에서 함수별 요약 메모리 상태 $\text{AbstractNodeMemMap}_f$ 로 대응하는 $\text{AbstractFunctionMem} : F \rightarrow m^\#$ 로 나타낸다.

요약실행 의미론은 CFG 노드 l 을 요약 메모리 상태 $M^\# : \text{AbstractVarMemMap}$ 에서 실행했을 때 후속으로 실행할 CFG 노드 l' 과 갱신된 요약 메모리 상태 $M'^\# : \text{AbstractVarMemMap}$ 를 반환하는 단일 실행 단계로 표현할 수 있으며, $(l, M^\#) \hookrightarrow^\# (l', M'^\#)$ 로 표기한다.

이후 l' 에 대한 갱신된 요약 프로그램 상태 $M'^\#$ 과 기존에 계산된 요약 프로그램 상태 $\text{AbstractNodeMemMap}(l')$ 과의 join연산 $M'^\# \sqcup \text{AbstractNodeMemMap}(l')$ 을 수행하여 l' 의 상태를 갱신한다.

따라서 요약 실행 알고리즘은 다음과 같이 설명할 수 있다:

1. 주어진 함수 f 에 대한 초기 상태 $S : \text{AbstractVarMemMap}$ 와 진입 노드 l_{init} 가 주어졌을 때 $(l_{\text{init}}, S) \hookrightarrow^\# (l', M'^\#)$ 을 계산
2. 기존의 $\text{AbstractNodeMemMap}(l')$ 를 $\text{AbstractNodeMemMap}(l') \sqcup M'^\#$ 로 갱신
3. 이후 l' 에서 바로 도달 가능한 후속 노드들 $\text{next}(l')$ 에 대해 위 과정을 반복해서 수행

위 알고리즘은 초기 프로그램 상태에서부터 각 노드에 대해 요약실행 의미론을 적용하여 노드별 상태를 계속해서 갱신하고, 더 이상 변화가 생기지 않는 고정점에 도달할 때 까지 반복 수행한다. 이렇게 계산된 고정점 프로그램 상태는 각 노드에서 관측 가능한 모든 값에 대한 정보가 반영된 프로그램 상태이며, 실제로 실행했을 때의 모든 값을 포함한 보수적인

 국민대학교 소프트웨어학부 다학제간캡스톤디자인I	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

$$\begin{array}{ccc}
 M^\# : \text{AbstractVarMap} & \xrightarrow{\hookrightarrow^\#} & M^{\#'} : \text{AbstractVarMap} \\
 \downarrow \gamma & & \downarrow \gamma \\
 \sigma : \mathbb{V} \rightarrow v & \xrightarrow{\hookrightarrow} & \sigma' : \mathbb{V} \rightarrow v
 \end{array}$$

그림 14: 요약 실행단계 $\hookrightarrow^\#$ 와 구체적 실행단계 $\hookrightarrow$ 과의 대응

근사 상태가 된다. 컴파일러가 data-flow analysis를 수행할 때 활용하는 Kildall 알고리즘[8]처럼 CFG위에서 노드별 상태를 계산하는 과정이 유사하다.

그림 14은 요약 실행의미론과 구체적 실행의미론의 관계를 묘사한다. 앞서 설명한 보수적인 근사가 성립하기 위해선, 요약 메모리 상태 $M^\#$ 가 모든 명령문에 대해 $M^{\#'}$ 로 요약실행이 된다면, $M^\#$ 의 구체적 상태 $\sigma = \gamma(M^\#)$ 에 대해 실제로 실행했을 때 얻는 상태 σ' 는 $M^{\#'}$ 의 구체적 상태 $\gamma(M^{\#'})$ 포함되어 있어야 한다.⁴

위 과정을 그대로 실행할 경우 잠재적으로 끝나지 않는 프로그램(무한루프가 포함된 프로그램 등)의 경우 고정점에 도달할 수 없기에 분석도 종료되지 않는다. 이는 $M^\#$ 에서 사용되는 요약도메인의 높이가 무한한 경우 새로 갱신되는 $\text{AbstractNodeMemMap}(l') \sqcup M^{\#'}$ 가 계속해서 증가하기 때문이다. 따라서 무한히 증가하는 요약상태에 대한 고정점을 제공하는 widening 연산 $\text{AbstractNodeMemMap}(l') \nabla M^{\#'}$ 을 적용하여 고정점에 도달할 수 있도록 유도해야 한다. TraceInspector에서 사용한 요약 도메인 중 정수 구간 도메인과 배열 배열 축약 도메인은 그 높이가 무한하기에, 위에서 정의한 widening을 적용하여 분석이 유한한 시간 내에 종료하도록 보장하였다.

최종적으로 TraceInspector에 대한 논리적 건전성을 다음과 같이 정의할 수 있다:

정리 (TraceInspector분석기의 논리적 건전성). *Imp*의 모든 프로그램 s 와 초기 프로그램 상태 σ 에 대해, $M^{\#'} = \text{analyze}(s, \sigma)$ 를 TraceInspector분석기가 계산한 최종 요약 고정점이라 하자. 이때 s 를 σ 에서 실행했을 때 도달 가능한 모든 구체적 프로그램 상태 σ' 에 대해

$$\sigma' \in \gamma(M^{\#'})$$

가 성립한다.

증명은 각 요약도메인과 타입에 대한 갈루아 연결을 보인 후 *Imp*의 명령문에 대한 구조적 귀납으로 실시한다.

이 증명은 TraceInspector의 분석 결과가 주어진 요약 의미론과 요약 도메인이 정의된 구체적 실행의미론에 대한 정확한 근사를 항상 취한다는 것을 보장한다.

⁴보다 엄밀한 정의는 메모리 상태 $\mathbb{V}$ 의 각 변수에 v 에 대한 pointwise-lifted 명세로 정의한다: $\forall v \in \mathbb{V}. \sigma[v] \in \gamma(M^\#[v])$

 국민대학교 소프트웨어학부 다학제간캡스톤디자인 I	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

오류 상태 판별

계산된 요약 상태 위에서 TracelInspector 분석기는 두가지 오류 조건을 검사한다. 첫 번째는 배열 인덱싱 $a[i]$ 가 주어졌을 때 가능한 모든 i 의 값들이 a 의 잠재적인 길이보다 작은지, 그리고 0보다 큰지 검사한다. 해당 검사는 요약 도메인상에서 실시하며 보수적인 추정이기 때문에 올바른 인덱싱임을 판정하면 모든 a 와 i 의 값에 대해 안전하지만, 반대로 오류가 나지 않을 $a[i]$ 에 대해서도 오류가 발생할 수 있음을 알리게 된다.

두번째 오류 상태는 CFG의 조건 노드에 대해 수행하며, 조건식의 요약도메인 계산값이 $\top$ 이 아닌 true, false, $\perp$ 인 경우 경고를 한다. 조건식의 요약 결과가 $\top$ 인 경우 특정 입력들에 대해서는 참 혹은 거짓임을 나타내기에 즉각적으로 조건식의 결과에 대해 알 수 없게 된다. 하지만 요약 고정점 상태에서 결과가 참 또는 거짓인 경우 해당 조건식에 도달했을 때 가능한 모든 값에 대해 참 또는 거짓으로 계산됨을 나타내므로 조건식 자체가 항진명제 혹은 모순명제가 되어 특정 분기가 도달 불가능함을 함의한다.

2.2.1.2 TracelInspector 프론트엔드

TracelInspector 분석기가 위와 같은 이론을 바탕으로 결과값을 내뱉어 준다. 하지만, 이 상태 만으로, 프로젝트가 추구하는 "알고리즘 시각화"라는 개념을 실현하기는 힘들다. 따라서, 분석기와 연계하여 분석기의 출력을 정의하고 그를 바탕으로 UI와 실효 기능을 만들게 되었다.

프론트엔드 개발의 다음과 같은 원칙(Code of Conduct)을 적용하였으며, 이를 최상위로 추구한다.

1. 복잡한 구조보다는 프로젝트가 추구하는 바를 실현하기 위해, KISS(Keep It Simple and Stupid) 철학을 준수한다. 2. 프론트엔드가 분석기인 만큼, 절대적인 안정성과 코드의 배포성을 지키기 위해 불필요한 라이브러리 사용은 지양한다. 3. 기능은 물론이고, 사소한 디자인 또한 모든 브라우저에서 동일하게 적용함을 원칙으로 한다.

UI 요약설명

프론트엔드 UI는 "React-tRace"와 유사하게 제작 되었으며, 추가 설명은 다음 소제목에서 다루겠다.

상단에는 분석기 조작버튼이 존재하며, 각각 파일 입력, 분석 개시, 자동 재생 기능으로 구성되어 있다. 중간에는 각각 소스코드의 내용을 띄워주는 창, 분석한 그래프를 띄워주는 창, 분석기가 잡은 오류나 경고를 띄우는 창으로 구성되어 있다. 하단에는 분석기가 분석한 과정을 미디어 플레이어 처럼 조작하여 확인할 수 있는 슬라이더 바가 존재하며, 방향키나 드래그, 자동 재생 기능으로 제어할 수 있다.

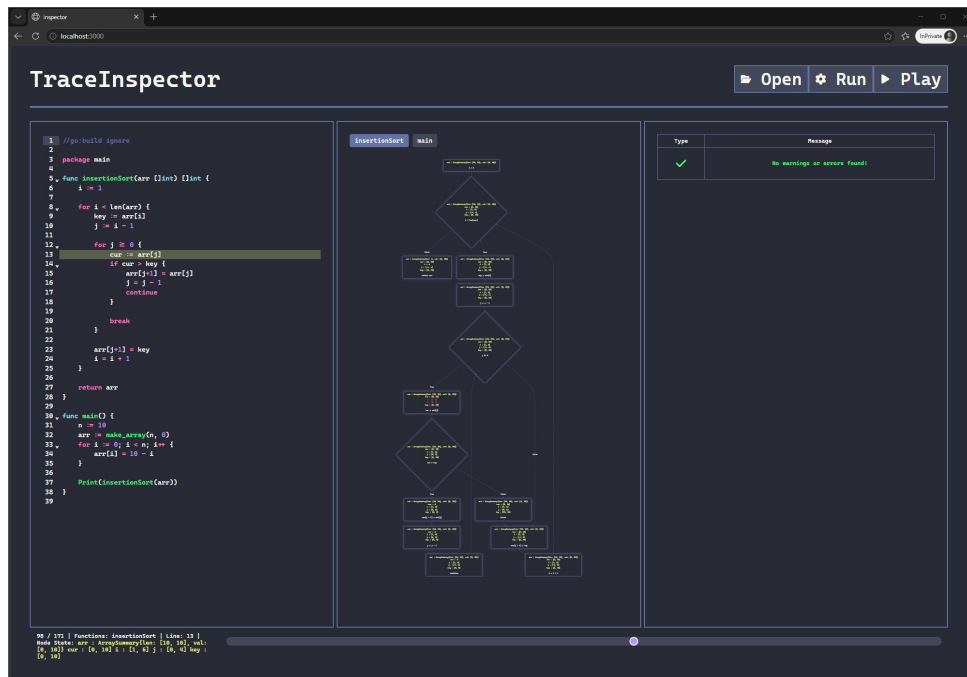


그림 15: TraceInspector전체 UI 구조

작동방식 상세설명

프론트엔드를 실행하면 위와 같은 창이 생성이 된다. 여기서 "Open" 버튼을 눌러 Go 소스코드를 불러온다.

위 기능을 위해, HTML의 기본으로 내장되어 있는 파일 입력을 사용하였고, 파일 입력이 성공하면, 왼쪽 코드 창에 출력한다.

다음으로, 분석을 위해 "Run" 버튼을 누르게 되면, 입력 되어 있는 소스코드가 front_src/go/source.go 경로에 저장된다. 이때, "Multer" 라이브러리를 이용하여 파일 시스템에 직접적으로 출력하고, 이 과정을 ExpressJS로 제작한 서버에 /upload 신호로 넘겨 처리한다.

처리가 완료가 되면, front_src/go/source.go를 이용하여 분석기를 돌리고, 이 과정은 NodeJS 라이브러리 child_process 내에 있는 exec 함수를 이용한다. 분석기를 돌리면 분석기의 분석 과정이 생략된 1차 그래프 생성을 위한 JSON 파일이 front_src/json/output.json 생성된다. 추후에 변형을 가할 때 용이하도록 분리해 놓은 것이다. 이 과정을 ExpressJS로 제작한 서버에 /run 신호로 넘겨 처리하고 ReactJS 클라이언트로 복귀한다.

이후, 분석 과정이 생략된 경우를 감안하여 1차로 MermaidJS 그래프를 생성하여, 가운데 그래프 화면에 띄워준다. 이 과정에서, 향후 분석 과정을 각 그래프의 적용해야 하기 때문에 Mermaid_t와 그 하위 클래스를 만들어 인스턴스로 저장하여준다.

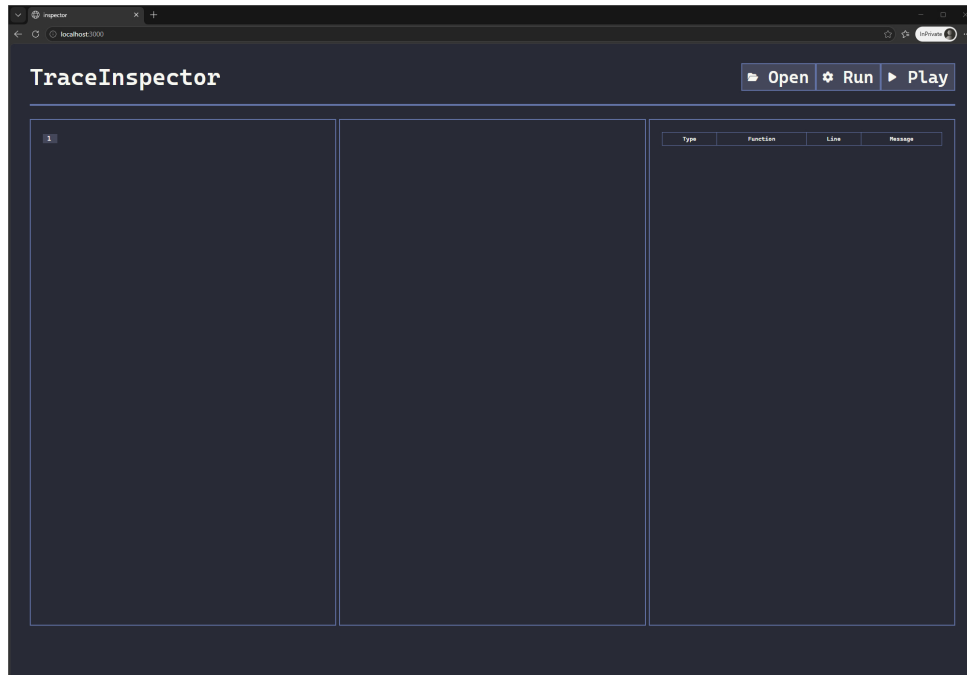


그림 16: 초기 UI

바로 분석 과정을 생성하기 위해, 곧장 ExpressJS로 제작한 서버에 /debug 신호로 넘겨 처리할 준비를 한다. 이 때, `front_src/go/source.go`를 이용하여 다른 플래그로 분석기를 돌리고, 이 과정은 NodeJS 라이브러리 `child_process` 내에 있는 `exec` 함수를 이용한다. 분석기를 돌리면 분석기의 분석 과정이 기록된 JSON 파일이 `front_src/json/debug.json` 생성되고, 이를 JSON 형식으로 클라이언트로 귀속한다. 귀속된 JSON 파일은 2차 MermaidJS 그래프를 제작을 위하여, `Debug_t` 클래스를 만들고 이를 배열로서 저장한다.

`Debug_t` 배열의 귀속된 인스턴스 중, 경고 혹은 오류와 같은 인스턴스는 오른쪽 창에 띄우도록 준비한다. 단, 분석 과정이 기록된 인스턴스들은 따로 다른 배열로 분리시킨 후, 각 변동 과정을 하이라이트한 MermaidJS 소스 코드 배열을 별도로 생성하여 준비한다. 이는, 분석 과정을 조작하여 살펴볼 때, MermaidJS가 유동적으로 변화할 수 없는, 컴파일 되어있는 SVG 형식으로 그래프를 출력하기 때문에, 컴파일을 하기 위한 목적과 함께, 슬라이더 구현도 `for-range`로 처리할 수 있다는 장점도 겸하고 있어 다음과 같은 방식을 채택하였다.

슬라이더 생성이 완료되면, 각 그래프와 디버그 창을 슬라이더와 연동시키고 Zoom 기능이나 Drag 기능을 위해 부가적인 요소를 초기화한다. 최종적으로, 모든 것들이 연동되어 있는 슬라이더와 2차 그래프를 생성하여 "Run" 연산을 모두 마친다.

이 모든과정은 "Run" 버튼 한번에 클릭으로 이루어지기에, 소스코드가 커지면 그의 비례하여 초기 소요시간이나 메모리 사용량이 증가할 수 있으나, 추가적인 갱신은 이미 완료한 상태이기에, 딜레이의 요소는 슬라이더 조정 간에 생길 수 있는 컴파일 시간이 전부이며,

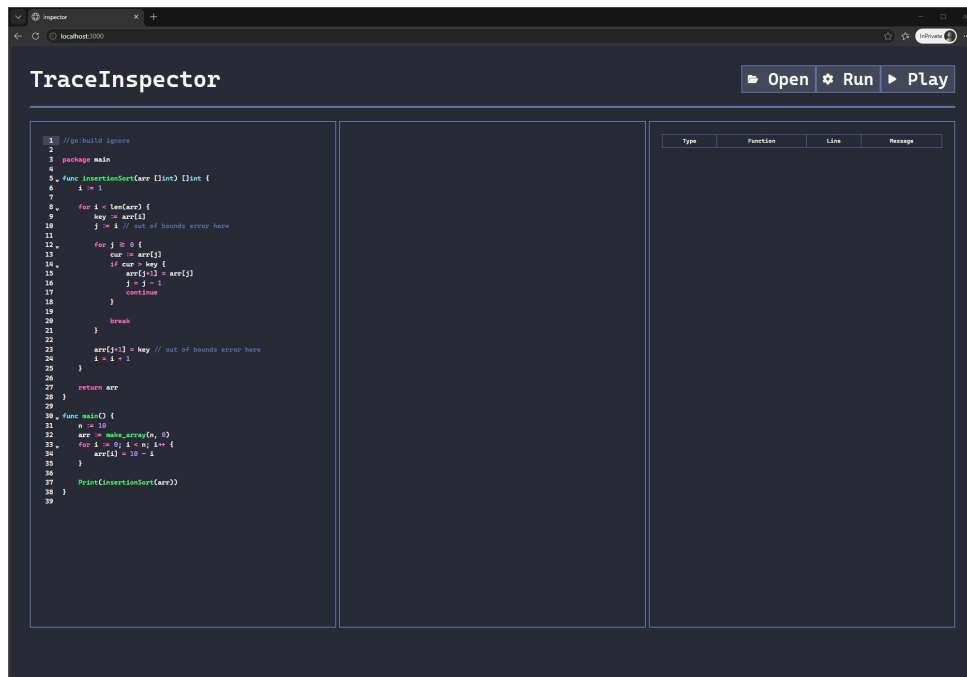


그림 17: Open 이후 UI

메모리 사용량은 증가하지 않는다.

각 변동사항은 그래프에 빨간색 글씨로 표기되며, 아래 하단에 삼각 점은 경고 및 오류가 발생한 위치를 찍어주는 점이다. 그래프를 조정하여 변동사항을 실시간으로 확인 할 수 있으며, "Play" 버튼을 눌러 자동화 할 수 있다.

사용요소 상세설명

위에서 잠깐 언급했지만, 이 UI를 구성하기 위해 들어간 요소들이 대거 있으며, 이를 간단히 소개하도록 하겠다.

1. React + TypeScript + Vite + ExpressJS: 각 요소는 프로그램 작동을 위한 핵심 프레임 요소들이며, 사용자 또한 반드시 각 요소를 설치하여야 동작한다. 2. Cascadia Mono with Nerd Font: 현 프론트에 사용되었던 글꼴이며, 아이콘 또한 현 글꼴에서 제공되는 아이콘을 사용하였다. 3. CodeMirror: 가운데 왼쪽에 있는 코드 에디터를 구성하기 위해 사용된 라이브러리다. 4. Multer: 직접적인 서버 내부 파일 시스템에 입출력을 수행하기 위해 사용된 라이브러리다. 5. Mermaid: 분석 과정 시각화를 위해 사용된 흐름차트 생성 라이브러리다. 6. panzoom: 그래프를 확대하거나 드래그 하여 탐색할 수 있도록 사용된 라이브러리다. 7. Go: 프론트 개발에는 사용되지 않았지만, 동작을 위한 CLI 분석기 제작을 위해 필요한 프로그래밍 언어다.

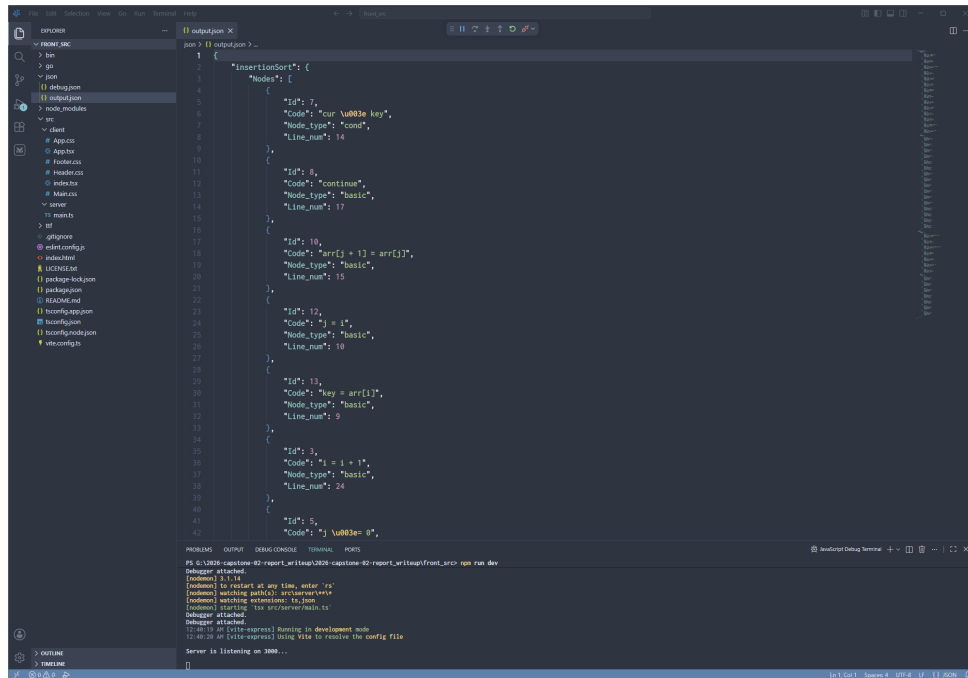


그림 18: 1차 그래프 JSON 파일

2.2.2 시스템 기능 요구사항

최종적으로 TraceInspector는 다음과 같은 통합 단계를 수행한다:

1. 웹 인터페이스를 통해 소스 파일 업로드
2. TraceInspector분석기를 소스 파일에 대해 호출
3. 소스 파일을 Imp로 변환
4. ImpAST를 CFG로 변환
5. CFG 위에서 요약 프로그램 상태 고정점 계산
6. 고정점 상태 위에서 오류 명제 판정
7. 계산된 요약 상태 시계열 및 오류 정보를 출력
8. 해당 정보를 웹 인터페이스에 렌더링
9. 사용자 조작에 따른 인터페이스 갱신

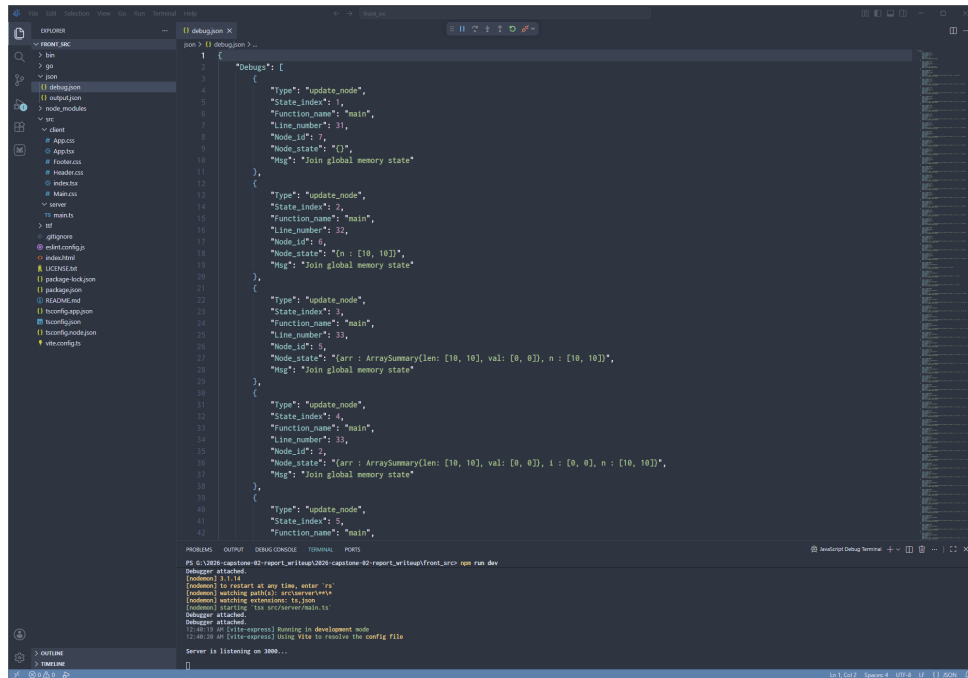


그림 19: 2차 그래프 JSON 파일

2.2.3 시스템 비기능(품질) 요구사항

TraceInspector 분석기의 절대적인 요구사항은 논리적 건전성 준수이다. 따라서 분석기를 한번에 구현하는 것이 아니라 단계적으로 구현하며 지속적인 코드 리뷰 및 건정성 검토를 통해 건전성을 추구하고자 하였다.

0단계는 $\mathbb{Z}_+$ 정수체계 및 구간도메인 구현 및 검증이었다. 해당 요소들은 분석기의 기반이 되는 요소들이고 산술 연산이 많이 포함된 만큼 구현 후 코드 리뷰 및 테스트를 통해 건정성을 확인하였다.

1단계는 Imp언어 중 대입 및 식 계산을 구현 목표로 하였다. 반복문과 조건문 등 제어 흐름을 배제하고 오로지 순차적인 명령에 대해서 구현을 하여 구체적 및 요약 의미론의 commutativity를 검증하고 논리적으로 건전한지 확인하였다.

2단계는 조건문과 배열 타입 지원이다. 조건문의 경우 루프 조건 필터링 함수를 구현해야 하기에 다시금 대수적 연산의 많은 구현을 필요로 해 면밀한 코드 리뷰 및 건정성 검토가 필요했고, 배열 타입은 요약도메인 선정 및 설계 시의 트레이드오프들을 고려해야 하기에 별도로 상당한 시간을 투자하였다.

3단계는 반복문과 재귀적 함수 호출이다. 두 경우 모드 "무한히 실행되는" 코드가 들어오는 요소들이며, 이에 따라 widening 연산(∇)과 함수 호출 단위 처리를 구현해야 한다. 특히나 widening 연산의 경우 잘못 구현할 경우 지금까지 쌓아온 논리적 건정성이 무너질 수 있기에 이론적인 배경을 바탕으로 구현 후 테스트를 많이 수행하였다.

도메인 및 분석기를 설계하는 과정에서 특정 도메인을 하드코딩 하지 않고 인터페이스

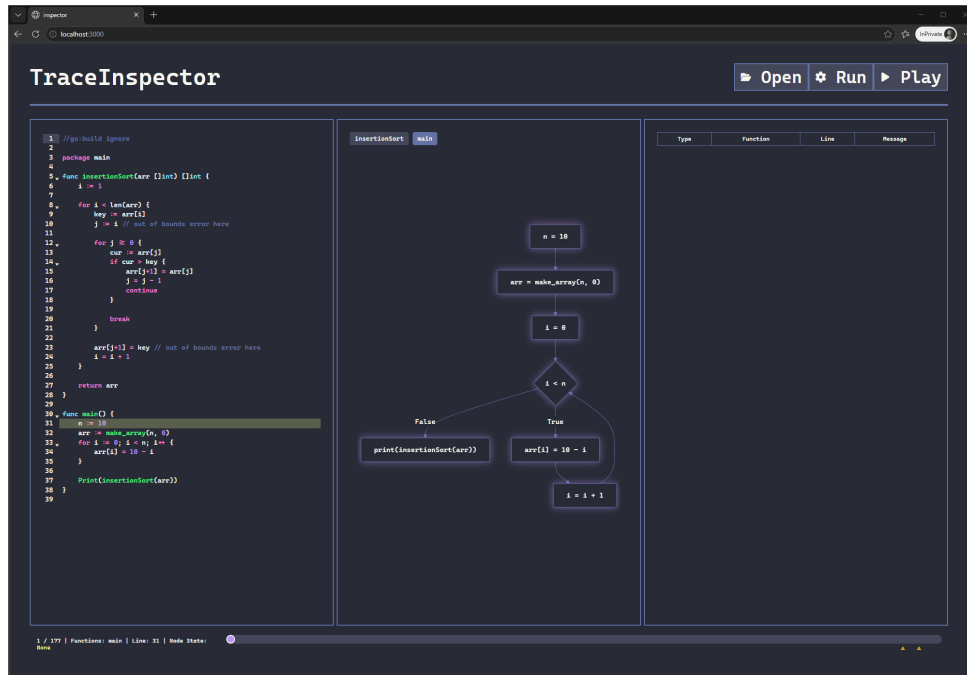


그림 20: Run 이후 UI

로 설계하여 향후 새로운 도메인 구현 및 통합이 가능하도록 설계하였다. 이러한 설계는 단순히 확장성을 넘어 도메인 구현 시 일관적인 구조를 요구하고, 모듈화한 테스트를 가능케 하여 코드의 품질 및 안전성에 크게 기여를 하였다. 특히나 요약도메인의 타입간 연산을 구별할 수 있게 되어 요약의미론 상에서 타입 오류가 발생하지 않도록 보장하는 이점을 누릴 수 있게 되었다.

 국민대학교 소프트웨어학부 다학제간캡스톤디자인I	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

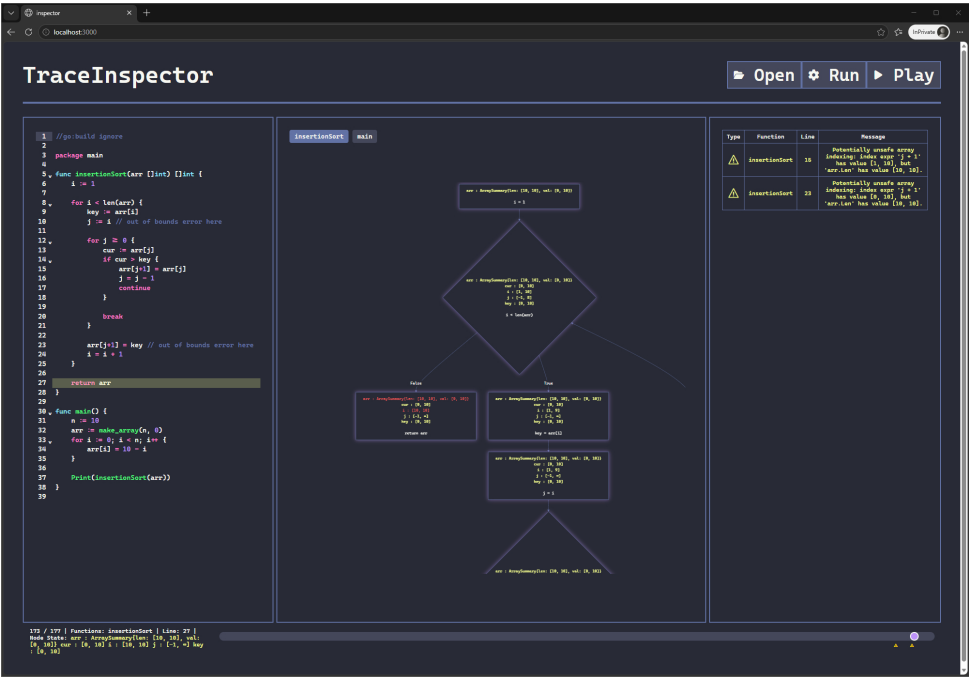


그림 21: 실행 중인 UI

 국민대학교 소프트웨어학부 다학제간캡스톤디자인I	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

2.2.4 시스템 구조 및 설계도

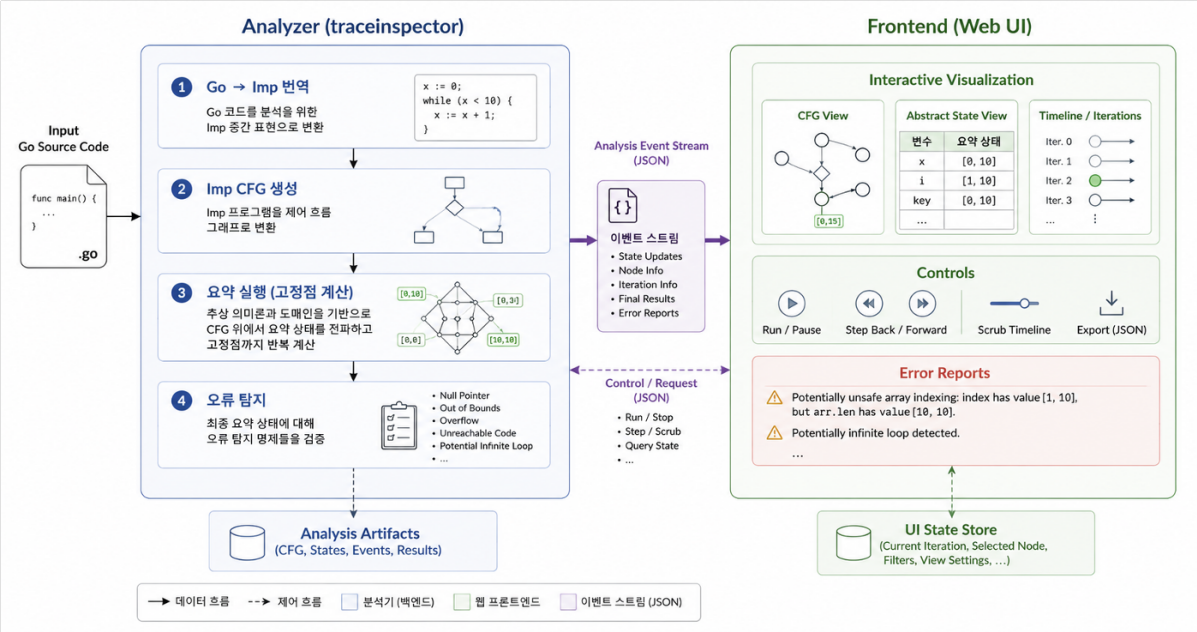


그림 22: TraceInspector전체 시스템 구조

그림 22는 TraceInspector의 전반적인 구조를 나타낸다. TraceInspector분석기는 입력 Go 프로그램에 대한 요약 프로그램 상태를 계산하여 오류 정보와 함께 JSON 형태로 표준 출력에 출력한다.

TraceInspector프론트엔드는 사용자로부터 코드를 입력받아 분석기를 호출하여 수행 결과를 시각화하는 웹 인터페이스와 백엔드로 구성되어 있다.

2.2.5 활용/개발된 기술

TraceInspector 분석기는 Go 소스 파일 파싱을 위한 go/parser[18] 패키지를 활용한다. 이후의 모든 분석 단계는 외부 라이브러리를 활용하지 않고 직접 구현되었지만 상술했듯이 요약해석 이론 자체는 잘 알려진 정적 분석 기법이다.

TraceInspector 분석기 UI는 다음과 같은 요소들을 사용한다.

1. React + TypeScript + Vite + ExpressJS: 각 요소는 프로그램 작동을 위한 핵심 프레임 요소들이며, 사용자 또한 반드시 각 요소를 설치하여야 동작한다.
2. Cascadia Mono with Nerd Font: 현 프론트에 사용되었던 글꼴이며, 아이콘 또한 현 글꼴에서 제공되는 아이콘을 사용하였다.
3. CodeMirror: 가운데 왼쪽에 있는 코드 에디터를 구성하기 위해 사용된 라이브러리다.
4. Multer: 직접적인 서버 내부 파일 시스템에 입출력을 수행하기 위해 사용된 라이브러리다.
5. Mermaid: 분석 과정 시각화를 위해 사용된 흐름차트 생성 라이브러리다.
6. panzoom: 그래프를 확대하거나 드래그 하여 탐색할 수 있도록 사용된

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

라이브러리다. 7. Go: 프론트 개발에는 사용되지 않았지만, 동작을 위한 CLI 분석기 제작을 위해 필요한 프로그래밍 언어다.

2.2.6 현실적 제한 요소 및 그 해결 방안

TraceInspector 분석기는 본질적으로 입력 언어인 Go의 일부분에 대해서만 분석을 수행할 수 있다. 우선적으로 포인터 자료형에 대한 분석을 지원하지 않는다. 요약 실행의미론과 요약 프로그램 상태에서는 포인터나 주소 등의 위치와 참조의 개념 없이 오직 단일 값으로만 간주하였고, 지금의 요약 프로그램 상태는 포인터 자료형에 대한 추론을 할 수 없다. 포인터 자료형에 대한 분석은 abstract heap라는, $\text{AbstractVarMemMap} : \mathbb{V} \rightarrow \text{AbstractValue}$ 가 아닌 주소 $\mathbb{A}$ 를 값으로 대응하는 $\text{AbstractHeap} : \mathbb{A} \rightarrow \text{AbstractValue}$ 를 구현하여 논해야 한다.

또한 추론 가능한 식의 구체적 형태에 대한 제한이 존재한다. TraceInspector는 $\pm x \pm y \odot C$ 형태의 대수식에 대한 논리적 추론 및 제약 역산을 수행할 수 있다. 하지만 구체적인 동작 형태는 x, y, C 에 대한 요약도메인 값을 산출하기에, 실제 x, y, C 간의 논리적 관계에 대해 알 수 없다.

```
while (x < len(arr)) {
    Print(arr[x - 1])
}
```

와 같은 코드가 주어졌을 때, 각 변수의 값을 계산하지 않고 x 는 항상 $\text{len}(\text{arr})$ 의 길이보다 작다는 관계형 정보를 추론할 수 있다. 이러한 관계형 정보를 표현하기 위해선 Octagon[13], Convex Polyhedra등의 관계형 도메인과 여러 도메인 사이의 정보를 보안할 수 있는 reduced product 도메인[6]을 구현하고 논리적 조건식에 대한 정보를 구하는 형태의 분석기가 필요하다. 따라서 TraceInspector는 관계형 정보에 대한 추론 능력에 대한 제한사항이 존재한다.

이러한 특성은 구현된 요약도메인의 정확도에 구애받기에, 향후 보다 정확하고 다양한 정보를 표현할 수 있는 요약도메인을 구현하고, TraceInspector 분석기와 통합하는 작업이 필요할 것이다.

마지막으로 현재 구현된 분석기는 입력 언어가 단일 Go파일로 제한되어 있다. 설계 자체는 타 언어 $\rightarrow \text{Imp}$ 로의 번역기로 확장 가능하지만, 단일 파일에 대한 분석이라는 한계는 여전히 존재한다. 이에 따라 입력 언어형태에 맞는 전처리기를 구현하여 모듈 단위의 분석을 지원할 수 있도록 개선이 필요하다.

재귀적인 함수 역시 분석의 정확도를 저하시킨다. 현재 분석기는 함수 단위로 분석을 수행하기에 새로운 함수 호출에 진입한 경우 함수의 인지값으로부터 함수 내부 상태에 대한 계산을 새롭게 수행해야 한다. 즉 필요한 시공간적 자원은 호출 스택의 깊이에 비례해서 증가하게 된다. 더욱 더 문제가 되는 점은 이러한 재귀적 호출의 경우 호출자의 프로그램 상태와 인자에 대한 논리적인 성질을 추론하지 않는다는 점이다. 재귀 함수의 경우 total



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

recursion 성질을 만족하면 재귀가 언젠가는 종료한다는 것이 알려져 있다. 즉 호출자의 프로그램 상태의 값들에 대해 인자들이 단조 감소하는 것이 보여진다면(decreasing argument), 재귀 함수의 호출 체인이 언젠가는 종료할 것이라는 고수준의 추론이 가능하다. 이처럼 함수 단위가 아닌 호출자와 피호출자 간의 정보에 대해 추론하는 intra-procedural 분석을 구현한다면 보다 정확한 추론이 가능할 것이다.

결론적으로 정확하고 신뢰할 수 있는 분석기를 만드는 문제는

1. 분석 대상 언어의 기능 중 어느 부분집합만을 구현할 것인지
2. 요약 도메인 선정 및 구현 시 시공간적 비용과 정확도의 트레이드오프를 어떻게 조율할 것인지
3. 얼마만큼의 하위 정보를 용납할 것인지

과 같은 디자인 결정에 크게 영향받는다. 특히 이러한 문제들은 최적의 정해가 주어진 것이 아니라 트레이드오프 관계에 해당되는 만큼 분석기가 사용되고자 하는 영역의 특성과 도메인 지식을 활용하여 설계되어야 한다. 이러한 문제는 실제 분석기 개발자들도 많은 시간과 연구를 통해 지속적으로 개선하고 있는 영역이다. 따라서 이번 프로젝트를 통해 정적 분석 이론은 구현 가능했지만, 실제로 쓸만한 분석기를 만드는 것은 이론을 넘어 경험과 노하우가 압도적으로 중요하다는 점을 깨닫게 되었다.

2.2.7 결과물 목록

TraceInspector의 소스는 루트 경로 내에 traceinspector 와 front_src 로 크게 나누어져 있다.

traceinspector 경로는 분석기 구현체를 담고 있으며, 다음과 같은 하위 경로로 세분화되어 있다.

algebra 경로에는 분석기에서 활용되는 확장된 정수체계 및 대수적 논리식 조작 체계를 수록하고 있다:

- extended_integers.go는 무한대로 확장한 정수체계 $\mathbb{Z}_{\infty}$ 를 나타내는 ExtInt 구조체와 관련 연산들을 구현한다.
- simple_prop.go는 $\pm x \pm y \odot C$ 형태의 대수식을 나타내는 SimpleProp 구조체와 수식 조작, 그리고 SimpleProp $\Leftrightarrow$ Imp식(expression)간의 변환을 구현한다.
- arith_normalizer.go는 $a_1x_1 + a_2x_2 + \dots + a_nx_n + C$ 형태의 다항식을 나타내는 Polynomial 구조체와 수식 조작, 그리고 마찬가지로 Imp식간의 변환을 구현한다.

cmd 경로에는 분석기의 실행파일 생성을 위한 진입점 및 명령줄 인터페이스를 구현한 main.go를 수록하고 있다.



domain 경로에는 TraceInspector분석기의 요약 도메인 추상 인터페이스 및 정수 구간과 Bool 요약도메인, 그리고 filter 함수 $\mathcal{F}_B$ 인터페이스 및 구현을 수록하고 있다:

- domain.go는 요약도메인의 추상 인터페이스 AbstractDomain와 정수 요약도메인의 추상 인터페이스 IntegerDomain를 구현한다.
- array_top_domain.go는 모든 배열을 T로 요약하는 약식 배열 요약도메인을 구현한다.
- bool_domain.go는 Bool 요약도메인 및 관련 연산을 구현한다.
- interval.go는 구간 요약도메인 및 관련 연산을 구현한다.
- filter_query.go는 filter 함수 구현을 위한 FilterQuery 구조체 및 논리식의 형태를 정의한다.
- filter.go는 SimpleProp로부터 FilterQuery를 구할 수 있는 변환 함수 및 대수적 조작 함수들을 구현한다.

imp 경로에는 Imp의 정의와 Go $\rightarrow$ Imp번역기, 그리고 Imp의 구체적 실행의미론을 구현한 인터프리터를 수록하고 있다.

- imp.go는 Imp의 문법과 타입 정의, AST 구조체들을 구현한다.
- interpreter.go는 Imp의 구체적 실행의미론을 구현한, Imp프로그램을 실행하는 인터프리터를 구현한다.
- go2imp.go는 Go AST를 ImpAST로 번역하는 번역기를 구현한다.

탐레벨 traceinspector 경로에는 다음과 같은 중요 소스 파일들이 존재한다:

- abstract_mem.go는 AbstractValue, AbstractVarMemMap, AbstractNodeMemMap, 그리고 함수별 요약상태 저장용 AbstractFunctionMem 을 구현한다.
- analyzer.go는 고정점 계산 및 Imp의 요약 실행의미론을 구현한 AbstractAnalyzer 구조체를 구현한다.
- array_domain.go는 배열 타입에 대한 요약도메인의 추상 인터페이스 ArrayDomain를 구현한다.
- array_summary_domain.go는 배열 원소 축약 요약도메인을 구현한다.
- cfgdefs.go는 CFG 노드 및 간선들과 관련된 정의를 구현한다.



- `create_cfg.go`는 ImpAST를 CFG로 번역하는 알고리즘을 구현한다.

`front_src` 경로는 분석기 UI를 담고 있으며, 다음과 같은 하위 경로로 세분화되어 있다.
`bin` 경로에는 UI에서 활용되는 CLI 분석기 바이너리가 생성되어 있어야 한다:

- `traceinspector.o`는 UNIX 계열에서 돌릴 수 있는 `build.sh` 스크립트를 실행 하였을 때, 생성되는 CLI 분석기 바이너리 이름이다.
- `traceinspector.exe`는 Windows 계열에서 돌릴 수 있는 `build.bat` 스크립트를 실행 하였을 때, 생성되는 CLI 분석기 바이너리 이름이다.

`json` 경로에는 CLI 분석기 바이너리가 서버에서 작동 시 만들어 주는 JSON 파일들이 생성되어 있어야 한다:

- `output.json`은 1차 MermaidJS 흐름 차트 생성을 위한 그래프 노드 및 엣지 정보가 출력된다.
- `debug.json`은 2차 MermaidJS 흐름 차트 생성을 위한 분석 과정 정보가 출력된다.

`ttf` 경로에는 분석기 UI가 사용하는 글꼴들이 존재한다:

- `CaskaydiaCoveNerdFontMono-*.ttf`는 유니코드 문자셋을 변형하여 많은 아이콘을 추가한 "Nerd Font"와 "Cascadia Mono"를 합친 글꼴이다.

`src` 경로에는 분석기 UI 제작에 사용된 소스코드를 담고 있으며, 다음과 같은 하위 경로로 세분화되어 있다.

`client` 경로에는 ReactJS 클라이언트 제작을 위해 사용한 소스 파일이 존재한다:

- `App.css`는 전체 UI에 적용되는 기본 스타일을 정의한다.
- `App.tsx`는 실질적인 UI 제작에 사용된 모든 함수 및 기능을 구현한다.
- `Footer.css`는 하단 UI 요소들에 적용되는 스타일을 정의한다.
- `Header.css`는 상단 UI 요소들에 적용되는 스타일을 정의한다.
- `index.tsx`는 `App.tsx` 파일과 `index.html` 파일을 연결해주는 파일이다.
- `Main.css`는 중간 UI 요소들에 적용되는 스타일을 정의한다.

`server` 경로에는 ExpressJS 서버 제작을 위해 사용한 소스 파일이 존재한다:

- `main.ts`는 클라이언트와 상호작용하는 모든 호출을 처리하는 함수 및 기능을 구현한다.

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

탐레벨 front_src 경로에는 다음과 같은 중요 소스 파일들이 존재한다:

- index.html는 최상단 HTML 문서이다.
- package.json는 분석기 UI 제작에 사용되는 npm 라이브러리를 기술한 파일들이다.
- eslint.config.js는 JavaScript 정적분석기 설정을 정의하는 파일이다.
- vite.config.ts는 Vite 핸들러가 사용하는 설정을 정의하는 파일이다.
- tsconfig*는 TypeScript 컴파일러가 사용하는 설정을 정의하는 파일들이다.

2.3 기대효과 및 활용방안

본 분석기는 기존의 상용화된 분석기처럼 효율적이고 정확성 관점에서 경쟁하고자 구현한 것이 아니라, 오히려 반대로 효율성을 희생하고 그 동작 과정을 노출시켜 관찰 가능한 정적 분석기를 구현하고자 하였다.

정적 분석을 사용해본 경험은 있지만 구체적인 원리에 대해 잘 알지 못하는 소프트웨어 개발자들에게는 정적 분석 기술이 어떤 방식으로 동작하고, 단순히 한번에 결과가 계산 되는 것이 아닌 반복적인 알고리즘임을 전달하기를 기대한다. 컴퓨터공학을 전공했다면 알고리즘 강의 때 복잡해보이는 알고리즘도 단계적으로 절차에 따른 연산을 수행한다는 점으로 분해해서 볼 수 있듯이 정적 분석도 마찬가지로 알고리즘을 알고자 한다. 또한 프로그램 분석이라는 기술과 이론을 소개하면서 올바른 코드란 무엇인지, 그리고 프로그램에 대해 어떻게 논증할 수 있는지에 대해 생각해볼 기회를 제공하고자 한다.

정적 분석 혹은 요약해석을 공부했거나 공부하고자 하는 경우 보다 구체적인 도움을 제공할 수 있을 것으로 계산된다. 대부분의 이론 교과서는 *프로그램의 요약 메모리 상태에 대해 요약 의미론이 정의한 전이 단계를 반복적으로 수행하며 고정점을 계산한다* 라는 형태로 분석 알고리즘을 소개한다. 물론 자명한 결과지만 간단한 예제를 넘어 실제로 어떻게 동작하는지에 대한 직관은 파악하기 어렵고 와닿는 설명이라고 보기는 어렵다. 고정점 계산이라는 개념을 상호작용 가능한 시계열 정보로 나열하여 직접 변화를 관찰할 수 있게 제공하고, 구체적인 그래프 위에서 의미론의 전이 과정과 상태 갱신 과정을 파악할 수 있음으로써 이론을 직관적으로 이해할 수 있도록 도움을 주는 보조 도구로서의 역할을 수행할 수 있을것이라 기대된다.

3 자기평가

TraceInspector분석기를 구현하게 되면서, 요약해석 알고리즘이 실제로 어떻게 고정점을 계산하고, 실시간으로 어떻게 프로그램 상태를 추정하는지 그 과정을 관찰할 수 있어서



프로젝트 명	TraceInspector	
팀 명	팀 2	
Confidential Restricted	Version 1.0	2026-06-16

매우 흥미로웠다. 이 프로젝트를 진행하기 전부터 요약해석 이론에 대해서는 알고 있었지만 단순히 이론적으로 "반복을 통해 고정점을 계산한다"는 사실을 기호로써만 터득했지 실제로 어떤 형태로 그 과정이 동작하는지, 그리고 왜 항상 고정점에 도달할 수 있을지에 대해서는 크게 관심을 가지지 않았다.

하지만 분석기를 구현하고 처음으로 프론트엔드와 연결하여 시각화한 결과를 보니 정말로 "이론이 살아 움직이는" 직접적인 결과를 볼 수 있어서 아름답게 느껴졌다. 특히나 단순하게 요약해석 분석기를 구현하고 끝나는 것이 아니라 시각화 인터페이스와 연결하여 보여주는 만큼, 두 구성 요소간의 통합을 설계하고 시계열 형태로 분석 결과를 생성하는 분석기를 만들었기에 좋은 경험이었다.

마지막으로 프로그램이 단순히 텍스트에 불과한게 아닌 분석과 논증의 대상으로 인식하게 되었고, 프로그램 분석과 언어론이 논리학과 수학의 근원적인 문제들과 직접적인 연관성을 가지고 있음을 체감하게 되었다. 프로그래밍이 대중화되고 "소프트웨어"라는 결과물을 만드는 것에 집중을 하다보니 프로그래밍이 무엇인지, 그리고 프로그램 자체가 무엇인지에 대해서 깊게 생각해볼 기회는 많지 않았다. 따라서 이번 프로젝트를 통해 "프로그램, 실행, 오류" 등의 의미에 대해 심도있게 탐구해볼 수 있어 만들어낸 산출물 이상의 가치가 있는 시간이었다고 스스로 생각한다.

프론트엔드 담당의 경우 초반에 많이 접하지 못한 부분을 도전하면서 시간 조절을 못하여 프론트 개발이 많이 더딘 점이 있었다. 또한 분석기의 출력을 프론트와 연동하는 과정과 UI 스타일을 짜는것도, 디자인 부분에서의 창의력을 발휘해야 하기에 많이 우왕자왕하는 모습을 보였다고 생각한다. 하지만 이러한 딜레이와 문제점을 해결하기 위해 여러 자료를 찾아 생각도 못했던 영감을 얻기도 하였다. 특히 여러 요소들을 다방면으로 연동해야 하기에 문제발생이 잦는 문제도 많아 이 점을 보고받기 용이하게 Github Issue를 적극적으로 사용해 보았으며, 여러 환경에서 연동하여도 문제가 없도록 각각의 가상머신을 구축하여 테스트를 진행하였다. 아이콘과 같은 Asset 사용도 일반 이미지 대신의 아이콘을 포함한 글꼴 사용해 관리가 수월해지게 개선하였으며 디자인 테마도 다양하게 찾아보고 적용해 심플함을 살리며 너무 단순하게 보이지는 않도록 많은 노력을 하였다. 이 과정에서 많은 문제해결 능력과 경험을 쌓게 되었다고 생각한다.

4 참고 문헌

References

- [1] V Alfred, S Monica, Sethi Ravi, Ullman Jeffrey D, et al. *Compilers Principles, Techniques*. Pearson, 2007.



- [2] Cristiano Calcagno and Dino Distefano. Infer: An automatic program verifier for memory safety of c programs. In *NASA Formal Methods Symposium*, pages 459–465. Springer, 2011.
- [3] Patrick Cousot. *Principles of abstract interpretation*. MIT Press, 2021.
- [4] Patrick Cousot and Radhia Cousot. Abstract interpretation: a unified lattice model for static analysis of programs by construction or approximation of fixpoints. In *Proceedings of the 4th ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 238–252, 1977.
- [5] Patrick Cousot, Radhia Cousot, Jérôme Feret, Laurent Mauborgne, Antoine Miné, David Monniaux, and Xavier Rival. The astrée analyzer. In *European Symposium on Programming*, pages 21–30. Springer, 2005.
- [6] Patrick Cousot, Radhia Cousot, and Laurent Mauborgne. The reduced product of abstract domains and the combination of decision procedures. In *International Conference on Foundations of Software Science and Computational Structures*, pages 456–472. Springer, 2011.
- [7] Jacques-Henri Jourdan, Vincent Laporte, Sandrine Blazy, Xavier Leroy, and David Pichardie. A formally-verified c static analyzer. *SIGPLAN Not.*, 50(1):247–259, January 2015.
- [8] Gary A Kildall. A unified approach to global program optimization. In *Proceedings of the 1st annual ACM SIGACT-SIGPLAN symposium on Principles of programming languages*, pages 194–206, 1973.
- [9] Florent Kirchner, Nikolai Kosmatov, Virgile Prevosto, Julien Signoles, and Boris Yakobowski. Frama-c: A software analysis perspective. *Formal aspects of computing*, 27(3):573–609, 2015.
- [10] Ted Kremenek. Finding software bugs with the clang static analyzer. *Apple Inc*, 8:2008, 2008.
- [11] Chris Lattner and Vikram Adve. Llvm: A compilation framework for lifelong program analysis & transformation. In *International symposium on code generation and optimization, 2004. CGO 2004.*, pages 75–86. IEEE, 2004.

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

- [12] Jay Lee, Joongwon Ahn, and Kwangkeun Yi. React-trace: A semantics for understanding react hooks: An operational semantics and a visualizer for clarifying react hooks. *Proc. ACM Program. Lang.*, 9(OOPSLA2), October 2025.
- [13] Antoine Miné. The octagon abstract domain. *Higher-order and symbolic computation*, 19(1):31–100, 2006.
- [14] Anders Møller and Michael I Schwartzbach. Static program analysis. *Notes. Feb*, 2012.
- [15] Benjamin C Pierce. *Types and programming languages*. MIT press, 2002.
- [16] Henry Gordon Rice. Classes of recursively enumerable sets and their decision problems. *Transactions of the American Mathematical society*, 74(2):358–366, 1953.
- [17] Xavier Rival and Kwangkeun Yi. *Introduction to static analysis: an abstract interpretation perspective*. MIT Press, 2020.
- [18] Go Team. go/parser. <https://pkg.go.dev/go/parser>, 2014.
- [19] The Coq Development Team. The coq proof assistant, June 2024.
- [20] Alan Mathison Turing et al. On computable numbers, with an application to the entscheidungsproblem. *J. of Math*, 58(345-363):5, 1936.

5 부록

5.1 사용자 메뉴얼

TraceInspector 웹 인터페이스에서 분석 대상 Go 소스 파일을 Open을 통해 분석을 요청한다. 이후 인터페이스는 업로드한 소스코드와 이에 대응되는 CFG, 그리고 오류 발생 여부를 표시한다.

사용자는 이후 하단의 재생 바를 드래그하여 각 분석 단계별 요약 프로그램 상태를 CFG 상에서 관측할 수 있으며, 좌/우 방향키를 이용하여 한 단계별 이동을 할 수 있다.

CFG는 각 함수 단위로 생성되며, 함수별 명령문과, 명령문이 실행될 프로그램 상태를 표시한다. 현 분석 단계에서 갱신된 프로그램 상태를 붉은색으로 음영 처리되어 변화를 직접 확인할 수 있다.

CFG 탐색 창은 드래그와 스크롤을 통해 이동 및 확대가 가능하며, 더블클릭을 통해 상태를 초기화할 수 있다.

 국민대학교 소프트웨어학부 다학제간캡스톤디자인	결과보고서		
	프로젝트 명	TraceInspector	
	팀 명	팀 2	
	Confidential Restricted	Version 1.0	2026-06-16

마지막으로 재상 바 하단의 노란색 화살표는 오류 상태가 최초로 탐지된 분석 단계를 나타낸다. 화살표를 클릭하면 해당 분석 상태로 이동하고, 오류 정보와 더불어 어떤 명령의 어떤 변수 상태에서 오류가 발생하게 되었는지 관찰 가능하다.

5.2 운영자 메뉴얼

TraceInspector 분석기를 127.0.0.1:3000 에서 활성화하기 위해, 다음과 같은 작업이 있어야 한다.

1. Go와 NodeJS를 설치한다.
2. 운영체제에 맞게 build.sh 혹은 build.bat을 실행한다.
3. front_src 폴더가 경로로 되게 쉘 터미널을 실행한다.
4. node_modules 또는 package-lock.json가 존재한다면 즉시 삭제한다. (단, 초기 설정에만 해당한다.)
5. npm install 명령을 실행하여, 환경 설치 및 라이브러리를 다운받는다.
6. npm run dev 명령을 실행하여, 서버를 개설한다.

5.3 배포 가이드

5.4 Imp 형식 의미론

해당 부록은 Imp에 대한 big-step operational semantics의 일부를 수록한다.

프로그램 상태 $\sigma : \mathbb{V} \rightarrow \mathbb{Z}$ 는 각 변수를 값으로 대응시킨다.

식(expression)의 경우 프로그램 상태 σ 에서 식 e 를 실행했을 때 값 v 로 계산됨을 $\sigma \vdash e \Downarrow v$ 로 표현한다.

함수의 경우 각 함수 이름을 인자명들과 함수 몸통 명령문의 쌍으로 대응시키는 전역 함수 대응 $F : FunName \rightarrow (\mathbb{V}^*, S)$ 로 표현된다.



$$\sigma \vdash x \Downarrow \sigma(x) \text{ E-VAR}$$

$$\sigma \vdash n \Downarrow n \text{ E-INT}$$

$$\sigma \vdash b \Downarrow b \text{ E-BOOL}$$

$$\sigma \vdash s \Downarrow s \text{ E-STRING}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 + e_2 \Downarrow n_1 + n_2} \text{ E-ADD}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 - e_2 \Downarrow n_1 - n_2} \text{ E-SUB}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 * e_2 \Downarrow n_1 n_2} \text{ E-MUL}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 / e_2 \Downarrow n_1 / n_2} \text{ E-DIV}$$

$$\frac{\sigma \vdash e \Downarrow n}{\sigma \vdash -e \Downarrow -n} \text{ E-NEG}$$

$$\frac{\sigma \vdash e_1 \Downarrow v_1 \quad \sigma \vdash e_2 \Downarrow v_2}{\sigma \vdash e_1 = e_2 \Downarrow (v_1 = v_2)} \text{ E-EQ}$$

$$\frac{\sigma \vdash e_1 \Downarrow n_1 \quad \sigma \vdash e_2 \Downarrow n_2}{\sigma \vdash e_1 < e_2 \Downarrow (n_1 < n_2)} \text{ E-LT}$$

$$\frac{\sigma \vdash e_1 \Downarrow b_1 \quad \sigma \vdash e_2 \Downarrow b_2}{\sigma \vdash e_1 \wedge e_2 \Downarrow (b_1 \wedge b_2)} \text{ E-AND}$$

$$\frac{\sigma \vdash e_1 \Downarrow b_1 \quad \sigma \vdash e_2 \Downarrow b_2}{\sigma \vdash e_1 \vee e_2 \Downarrow (b_1 \vee b_2)} \text{ E-OR}$$

$$\frac{\sigma \vdash e \Downarrow b}{\sigma \vdash \neg e \Downarrow \neg b} \text{ E-NOT}$$

$$\frac{\sigma \vdash e_1 \Downarrow a \quad \sigma \vdash e_2 \Downarrow i}{\sigma \vdash e_1[e_2] \Downarrow a[i]} \text{ E-INDEX}$$

$$\frac{\sigma \vdash e \Downarrow a}{\sigma \vdash \text{len}(e) \Downarrow |a|} \text{ E-LEN}$$

$$\frac{\sigma \vdash e_1 \Downarrow v_1 \quad \dots \quad \sigma \vdash e_n \Downarrow v_n \quad F(f) = (x_1, \dots, x_n, S) \quad \langle S, [x_1 \mapsto v_1, \dots, x_n \mapsto v_n] \rangle \Downarrow \langle \sigma', \text{return}(v) \rangle}{\sigma \vdash f(e_1, \dots, e_n) \Downarrow v} \text{ E-CALL}$$

명령문 S 를 프로그램 상태 σ 에서 실행했을 때는 결과 상태 σ' 와 제어 흐름 결과 r 를 반환하며 $\langle S, \sigma \rangle \Downarrow \langle \sigma', r \rangle$ 로 표기한다.

제어 흐름 결과는 break, continue, return를 처리하기 위한 내부 상태이며 normal, break, continue, return(v)로 정의된다.



$$\frac{\langle \epsilon, \sigma \rangle \Downarrow \langle \sigma, \text{normal} \rangle \text{ SEQ-NIL} \quad \frac{\langle s, \sigma \rangle \Downarrow \langle \sigma_1, \text{normal} \rangle \quad \langle S, \sigma_1 \rangle \Downarrow \langle \sigma_2, r \rangle}{\langle s; S, \sigma \rangle \Downarrow \langle \sigma_2, r \rangle} \text{ SEQ-CONS}}{\langle s; S, \sigma \rangle \Downarrow \langle \sigma_1, r \rangle} \text{ SEQ-ABORT}$$

$$\frac{\langle s, \sigma \rangle \Downarrow \langle \sigma_1, r \rangle \quad r \neq \text{normal}}{\langle s; S, \sigma \rangle \Downarrow \langle \sigma_1, r \rangle} \text{ SEQ-ABORT} \quad \langle \text{skip}, \sigma \rangle \Downarrow \langle \sigma, \text{normal} \rangle \text{ S-SKIP}$$

$$\frac{\sigma \vdash e \Downarrow v}{\langle x := e, \sigma \rangle \Downarrow \langle \sigma[x \mapsto v], \text{normal} \rangle} \text{ S-ASSIGN}$$

$$\frac{\sigma \vdash e \Downarrow \text{true} \quad \langle S_t, \sigma \rangle \Downarrow \langle \sigma', r \rangle}{\langle \text{if } e \text{ then } S_t \text{ else } S_f, \sigma \rangle \Downarrow \langle \sigma', r \rangle} \text{ S-IFTRUE}$$

$$\frac{\sigma \vdash e \Downarrow \text{false} \quad \langle S_f, \sigma \rangle \Downarrow \langle \sigma', r \rangle}{\langle \text{if } e \text{ then } S_t \text{ else } S_f, \sigma \rangle \Downarrow \langle \sigma', r \rangle} \text{ S-IFFALSE}$$

$$\frac{\sigma \vdash e \Downarrow \text{false}}{\langle \text{while } e \text{ do } S, \sigma \rangle \Downarrow \langle \sigma, \text{normal} \rangle} \text{ S-WHILEFALSE}$$

$$\frac{\sigma \vdash e \Downarrow \text{true} \quad \langle S, \sigma \rangle \Downarrow \langle \sigma_1, \text{normal} \rangle \quad \langle \text{while } e \text{ do } S, \sigma_1 \rangle \Downarrow \langle \sigma_2, r \rangle}{\langle \text{while } e \text{ do } S, \sigma \rangle \Downarrow \langle \sigma_2, r \rangle} \text{ S-WHILETRUE}$$

$$\frac{\sigma \vdash e \Downarrow \text{true} \quad \langle S, \sigma \rangle \Downarrow \langle \sigma', \text{break} \rangle}{\langle \text{while } e \text{ do } S, \sigma \rangle \Downarrow \langle \sigma', \text{normal} \rangle} \text{ S-WHILEBREAK}$$

$$\frac{\sigma \vdash e \Downarrow \text{true} \quad \langle S, \sigma \rangle \Downarrow \langle \sigma_1, \text{continue} \rangle \quad \langle \text{while } e \text{ do } S, \sigma_1 \rangle \Downarrow \langle \sigma_2, r \rangle}{\langle \text{while } e \text{ do } S, \sigma \rangle \Downarrow \langle \sigma_2, r \rangle} \text{ S-WHILECONTINUE}$$

$$\langle \text{break}, \sigma \rangle \Downarrow \langle \sigma, \text{break} \rangle \text{ S-BREAK} \quad \langle \text{continue}, \sigma \rangle \Downarrow \langle \sigma, \text{continue} \rangle \text{ S-CONTINUE}$$

$$\frac{\sigma \vdash e \Downarrow v}{\langle \text{return } e, \sigma \rangle \Downarrow \langle \sigma, \text{return}(v) \rangle} \text{ S-RETURN}$$

5.4.1 요약 도메인 Go 인터페이스

TraceInspector분석기는 Go의 generics를 활용하여 특정 요약도메인 구현에 의존하지 않



```
type AbstractAnalyzer [IntDom domain.IntegerDomain[IntDom], ArrDom ArrayDomain[IntDom, ArrDom]] struct {
  Function_cfgs      FunctionCFGMap
  Function_defs      imp.ImpFunctionMap
  Settings           AnalysisSettings
  Intdomain_default  IntDom
  Arraydomain_default ArrDom
  function_pre_mem_map map[imp.ImpFunctionName]*AbstractFunctionMem[IntDom, ArrDom]
  output_handler     *AnalyzerOutputHandler
}
```

그림 23: AbstractAnalyzer 구조체 정의와 generics를 활용한 모듈러한 설계

고, 사용자가 정의한 임의의 요약도메인을 활용할 수 있도록 구현하였다. AbstractDomain 인터페이스를 설계하여 정수 및 배열 타입에 대한 요약 도메인을 정의하고 분석기에 바로 활용할 수 있도록 구현되어 있다.

코드 23는 Tracelnspector분석기를 초기화하는 구조체의 정의이다. IntDom, ArrDom에 IntegerDomain, ArrayDomain 를 만족하는 도메인 구현체를 넘겨주면 분석기는 구현에 구현받지 않고 온전하 동작할 수 있다.

```
type AbstractDomain [DomainImpl any] interface {
  IsBot() bool
  IsTop() bool
  CreateBot() DomainImpl
  CreateTop() DomainImpl
  Incl(DomainImpl) bool
  Join(DomainImpl) (DomainImpl, bool)
  Widen(DomainImpl) DomainImpl
  String() string
  Clone() DomainImpl
}

type IntegerDomain [DomainImpl any] interface {
  AbstractDomain [DomainImpl]
  From_IntLitExpr(imp.IntLitExpr) DomainImpl
  Add(DomainImpl) DomainImpl
  Sub(DomainImpl) DomainImpl
  Mul(DomainImpl) DomainImpl
  Div(DomainImpl) DomainImpl
  Mod(DomainImpl) DomainImpl
  Eq(DomainImpl) BoolDomain
  Neq(DomainImpl) BoolDomain
  LessThan(DomainImpl) BoolDomain
  GreaterThan(DomainImpl) BoolDomain
  Leq(DomainImpl) BoolDomain
  Geq(DomainImpl) BoolDomain
  Neg() DomainImpl
  Filter(FilterQueryType, DomainImpl) DomainImpl
}
```

그림 24: AbstractDomain 과 IntegerDomain 인터페이스 정의

코드 24는 각각 공통 요약도메인과 정수 요약도메인의 추상 인터페이스 정의이다. AbstractDomain 는 모든 요약도메인 구현체들이 구현해야할 핵심 함수들을 정의하고 있고, IntegerDomain 는 산술 및 비교 연산 등 정수를 나타내는 도메인이 구현해야 할 함수들을 명세하고 있다. 이처럼 해당 함수들을 구현한 구현체를 Tracelnspector분석기는 그 구체적 형태에 대해 신경쓰지 않고 요약 실행의미론과 접목해서 실행할 수 있는 구조로 설계되었다.

TraceInspector - 추적 가능한 시각화 기반 코드 정적분석기

배경

정적 분석(Static Analysis)은 프로그램을 실행하지 않고 코드의 실행 특성을 추론하여 잠재적인 오류를 탐지하는 기술이다. IDE, 컴파일러, CI 등 다양한 개발 과정에서 코드 품질과 안정성을 높이기 위해 활용되고 있음.
기존 분석기들은 분석 결과만 제시할 뿐 어떻게 오류가 탐지되었는지 파악하기 어렵고, 분석 과정을 일종의 “블랙박스”처럼 받아들이고 결과를 맹신하게 됨.

TraceInspector 소개

TraceInspector는 요약해석(Interpretation) 이론을 바탕으로 한 정적 분석기와 더불어 분석 과정을 시각적으로 보여주는 인터페이스로 구성되어 있음.
분석 중 생성되는 요약 상태(Abstract State)의 변화 과정을 사용자에게 실시간으로 보여주어 각 프로그램 지점에서 정보가 어떻게 전파 및 정제되는지를 직관적으로 관찰할 수 있도록 구현함.

분석 과정

1: Go 코드 → 중간 표현 언어(Imp)

Go AST를 간단한 형태의 Imp 언어로 번역. 분석기는 Imp 코드에 대해 동작하며, 분석기 구현 및 타 언어 이식 시 용이하도록 설계.

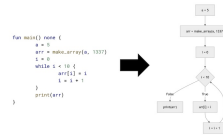
```
func main() {  
    a := 5  
    arr := make_array(a, 1037)  
    for i := 0; i < 10; i++ {  
        arr[i] = i  
    }  
    Print(arr)  
}
```



```
fun main() none {  
    a = 5  
    arr = make_array(a, 1037)  
    i = 0  
    while i < 10 {  
        arr[i] = i  
        i = i + 1  
    }  
    print(arr)  
}
```

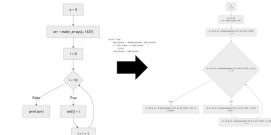
2: Imp → 제어 흐름 그래프(CFG)

프로그램 내 분기, 반복 등의 제어 흐름 단위로 나타내는 제어 흐름 그래프 형태로 표현. 각각의 명령문들은 단일 노드로 표현됨.



3: CFG 위에서 고정점 근사

CFG 노드별 진입 시점에서의 프로그램 상태를 근사. 고정점에 도달할 때까지 반복적으로 요약실행 및 상태 join/widening 수행.



분석 예시: 무한 루프 및 Dead Code Detection

```
1 func main() {  
2   a := 1  
3   for a > 0 {  
4     a++  
5   }  
6   Print(a)  
7 }
```

무한히 반복되는 코드의 경우 계산된 상태에 대해 widening 기법을 적용하여 분석이 종료할 수 있도록 유도. 이후 filtering을 통해 결과를 정제하여 정확도 손실을 방지.
추가로 항진/모순명제를 탐지하여 도달할 수 없는 코드에 대해 경보.

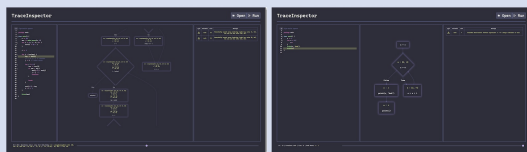
분석 예시: 배열 인덱싱 범위

```
1 i := 1  
2 for i < len(arr) {  
3   key := arr[i]  
4   arr[i+1] = arr[i]  
5 Print(i)
```

i: [1, 1]
i: [1, 10], len(arr): [10, 10]
i: [1, 9]
i: [1, 9], key: [0, 10]
i: [10, 10], key: [0, 10]

배열의 길이와 인덱스 변수에 대한 요약상태를 계산 후 안전한 인덱싱 범위인지 확인.

시각화 및 인터페이스



- 분석 과정에서 생성되는 상태 변화를 단계별로 재생 가능하도록 구현.
- 함수별 및 단계별 상태를 자유롭게 탐색 가능.

한계 및 향후 계획

- 관계형 요약도메인 : $i < \text{len}(\text{arr})$ 등과 같이 구체적인 값이 아닌 변수간의 논리적 관계를 나타낼 수 있는 대수적 도메인.
- 분석 정확도 개선 : reduced product domain, narrowing 구현 및 SimpleProp 체계 확장.
- Pointer inference : 포인터 및 메모리 동작 추상화를 통한 포인터 연산 검증.
- C → Imp, Rust → Imp 등 Imp를 활용한 다양한 언어 지원.
- 보다 다양한 언어 기능 지원 : 확장된 타입 체계 등.

결과보고서 발표자료

TraceInspector

추적 가능한 시각화 기반 코드 정적분석기

김신영, 강민성

코드 정적 분석이란?

- 프로그램을 실행하지 않고 코드를 논리적으로 분석하여 동작 특성을 분석하는 기법
- 각종 런타임 오류 탐지, 요구사항 준수 등 코드에 대한 조건을 검사
- IDE, CI 등 소프트웨어 개발의 다양한 단계에서 활용

```
infer@facebook:~/infer/examples$ infer -- javac Infer.java
Starting analysis (Infer version v0.5.0)
Computing dependencies... 100%
Analyzing 1 cluster, 100%
Analyzed 3 procedures in 1 file

Found 1 issue
Infer.java:12: error: NUL_DEREFERENCE
object s last assigned on line 11 could be null and is dereferenced at line 12
10.   int mayCauseNPE() {
11.       String s = mayReturnNull(0);
12. >   return s.length();
13.   }
14.
```

```
▼ App.tsx front_src/src/client 2
  ▲ 'act' is declared but its value is never read. ts(6133) [Ln 7, Col 39]
  ▲ 'TransformOrigin' is declared but its value is never read. ts(6133) [Ln 16, Col 24]
▼ create_cfg.go traceinspector 1
  ○ unused parameter: cond_node_loc unusedparams(unusedparams) [Ln 72, Col 53]
```

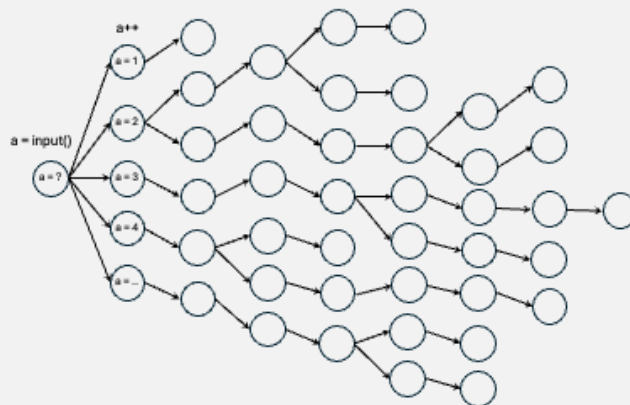
기존 분석기들이 답해주지 못하는 질문들

- 분석기가 프로그램 오류를 어떻게 탐지할까?
- 이 분석기의 결과를 완전히 신뢰할 수 있을까?
- 근원적으로 프로그램을 실행한다는 것의 구체적 의미는 무엇일까?
- 오류의 의미는?

⇒ 분석기를 오류 정보만 보여주는 밀폐된 프로그램으로 간주하지 말고 동작 원리를 보여주자!
⇒ 프로그램을 잘 짜는 것을 넘어 프로그램 자체를 논증의 대상으로 바라보는 관점을 소개하자!

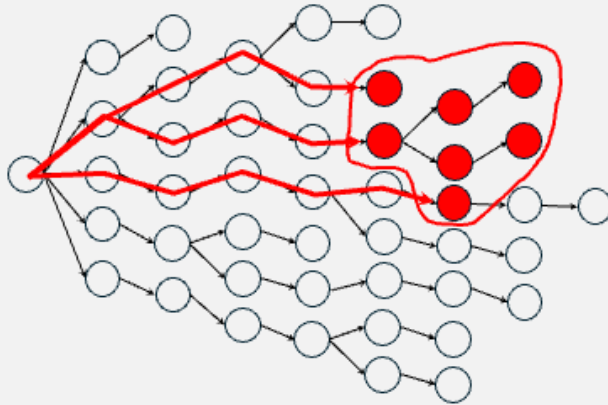
배경: 프로그램 실행 의미론

- 프로그램 흔적(trace): 실행 시 발생할 수 있는 프로그램 실행 경로(메모리 상태의 전이)



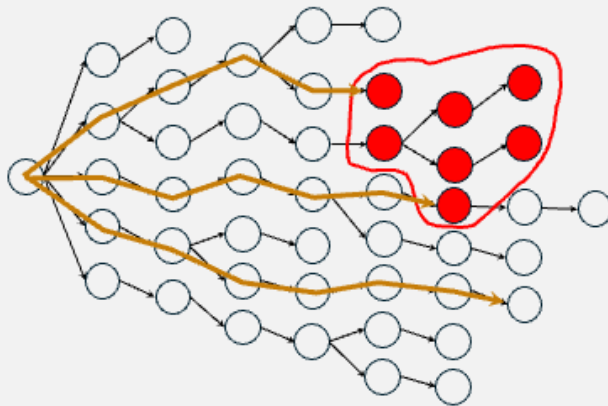
배경: 프로그램 실행 의미론

- 오류 흔적: 오류가 발생한 프로그램 상태가 포함된 실행 경로



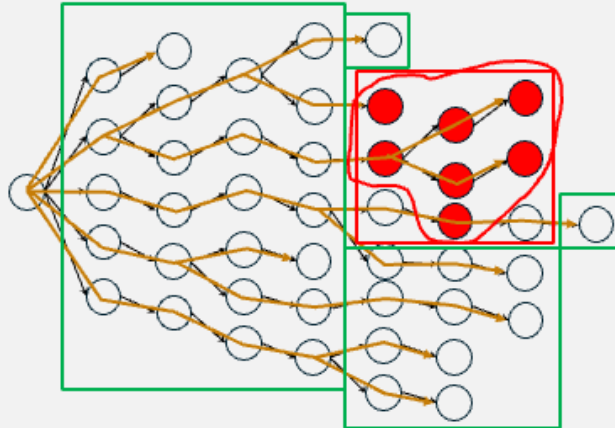
배경: 프로그램 실행 의미론

- 테스트: 특정한 프로그램 흔적을 검증 가능(모든 흔적 검증 불가능)



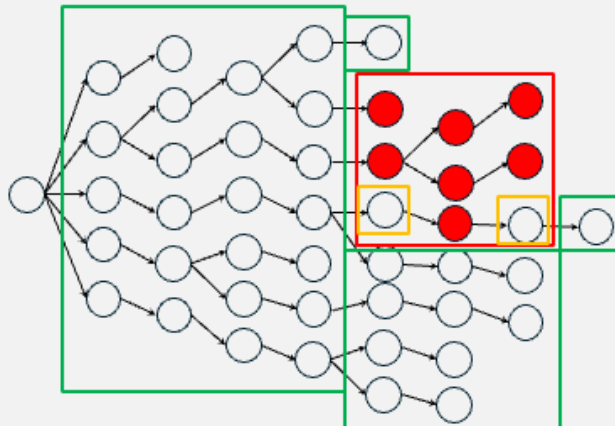
요약해석(Abstract Interpretation)

- 구체적 상태(concrete state)들을 “덜 정확하게” 표현한 요약 상태(abstract state)로 근사



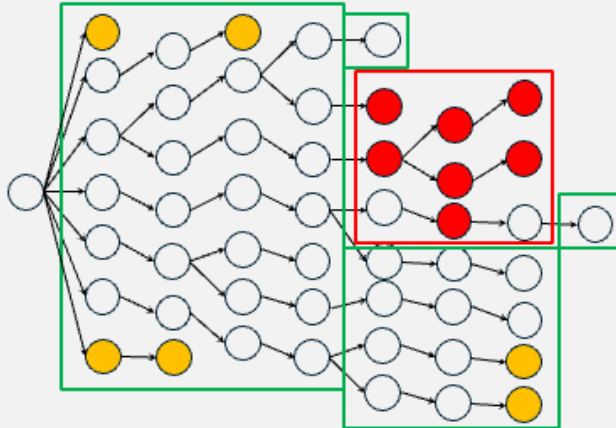
요약해석(Abstract Interpretation)

- 과다추정(over-approximation): “덜 정확하게” 근사했기에 오류가 아닌 상태도 오류로 간주함



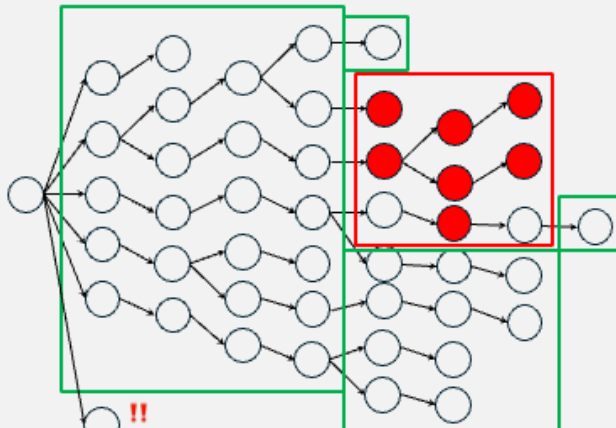
요약해석(Abstract Interpretation)

- 과다추정(over-approximation): 실제로 발생하지 않을 구체적 상태들도 반영하게 됨(보수적 근사)



요약해석(Abstract Interpretation)

- Soundness(논리적 건전성): 모든 구체적 상태는 빠짐없이 요약 상태에 반영되어야 한다

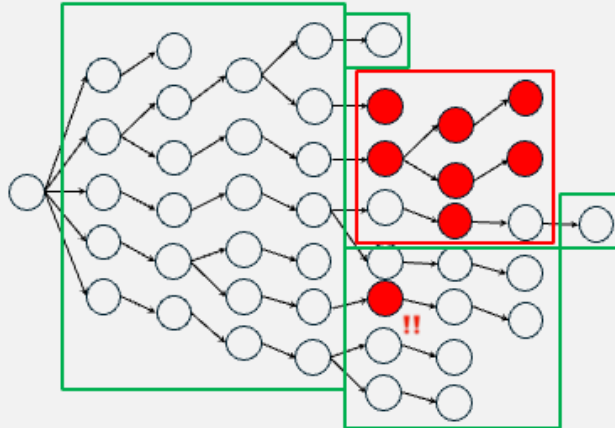


!!

실제로 가능한 상태를 놓치면 안됨

요약해석(Abstract Interpretation)

- Soundness(논리적 건전성): 모든 구체적 오류 상태는 요약 상태상으로도 오류여야 한다



요약해석(Abstract Interpretation)

- 요약해석의 보증: 요약상태로 근사된 프로그램에 대해 특정 오류가 존재하지 않음이 보여지면, 실행 가능한 모든 구체적 상태에 대해 해당 오류가 발생하지 않는다.
- 하지만 실제로 오류가 아닌 상태에 대해 오류로 판정하는 허위 경보(false positive)가 가능함
- 수학적으로 증명된 강력한 보장(galois connection, soundness property)

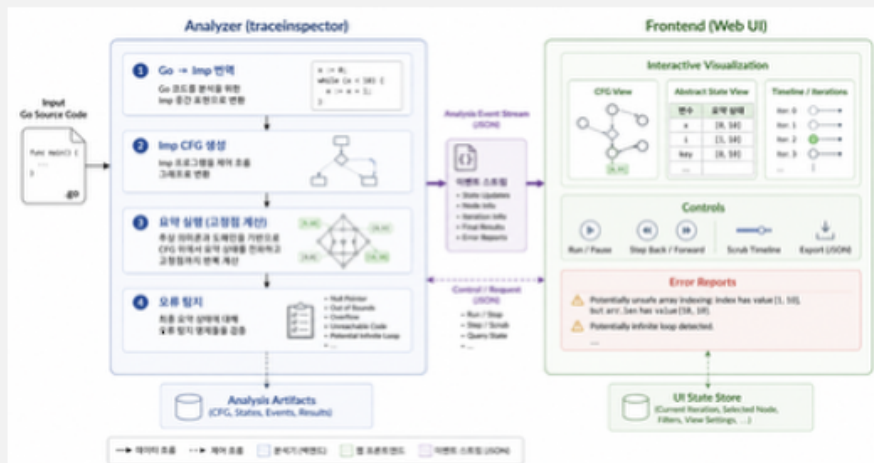
- The implication $\alpha(P) \sqsubseteq \bar{P} \Rightarrow P \sqsubseteq \gamma(\bar{P})$ holds when $\bar{P} = \alpha(P)$ so $P \sqsubseteq \gamma(\alpha(P))$, meaning that $\alpha(P)$ is a sound overapproximation of the concrete property P and
- The converse implication $P \sqsubseteq \gamma(\bar{P}) \Rightarrow \alpha(P) \sqsubseteq \bar{P}$ means that for any sound overapproximation $\bar{P}$ of the concrete property P , $\alpha(P)$ is stronger, that is, more precise than $\bar{P}$.

TraceInspector 소개

- 요약해석 이론을 구현한 코드 정적분석기
 - 배열 범위 검사 및 오류 탐지
 - 잠재적 무한루프 및 도달 불가능 조건 분기 탐지
 - 논리적 건전성 보장
- 요약상태 계산 및 변화를 관찰할 수 있는 *시각적, 반응형* 인터페이스
 - 오류 경보 전달뿐만 아니라 상호작용 가능한 인터페이스
- "어떤" 오류가 발생했는지 아니라 **"어떻게"** 오류가 탐지되었는지 정보 제공
 - **다양한 종류의 오류를 신속하게 분석하자!**
- 정적 분석 기술에 대한 친숙도 및 이해도 증가
- 프로그래밍 언어와 연관되어 있는 논리와 이론 소개

TraceInspector 구조

- 요약해석 분석기와 웹 인터페이스로 구성



TraceInspector 구조

- Go 입력 언어 → Imp 중간 언어로 번역
- 타 언어에 대한 Imp 번역기 구현 시 분석기 그대로 활용 가능

```

func main() {
    a := 5
    arr := make_array(a, 1007)
    for i := 0; i < 10; i++ {
        arr[i] = i
    }
    Print(arr)
}
    
```

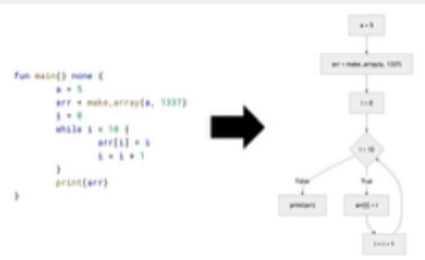
→

```

fun main() none {
    a = 5
    arr = make_array(a, 1007)
    i = 0
    while i < 10 {
        arr[i] = i
        i = i + 1
    }
    print(arr)
}
    
```



- Imp 프로그램에 대한 제어 흐름 그래프(CFG) 생성
- 각각의 명령문(statement)을 하나의 정점으로 표현
- 조건에 따른 흐름 분기



TraceInspector 구조

- 노드별 실행 전 프로그램 상태(precondition) 계산
- 요약 도메인으로 상태 근사
 - 정수: 구간 산술도메인으로 요약
 - 정수 배열: 배열 대표값 및 배열 길이를 정수 도메인으로 요약
 - boolean: boolean 격자로 요약
- 고정점(fixed-point) 계산
 - 더 이상 근사 결과가 변하지 않을 때까지 반복 보완
 - 분석 과정은 반복적인 흐름!



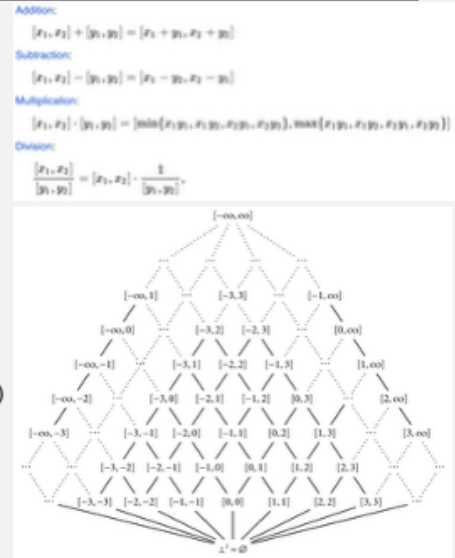
$$f^Q \quad f^\infty = \bigcup_i f^i = f(f^\infty)$$

$f^0 = \perp$
 f^1
 f^2
 f^3
 $\vdots$
 f^Q

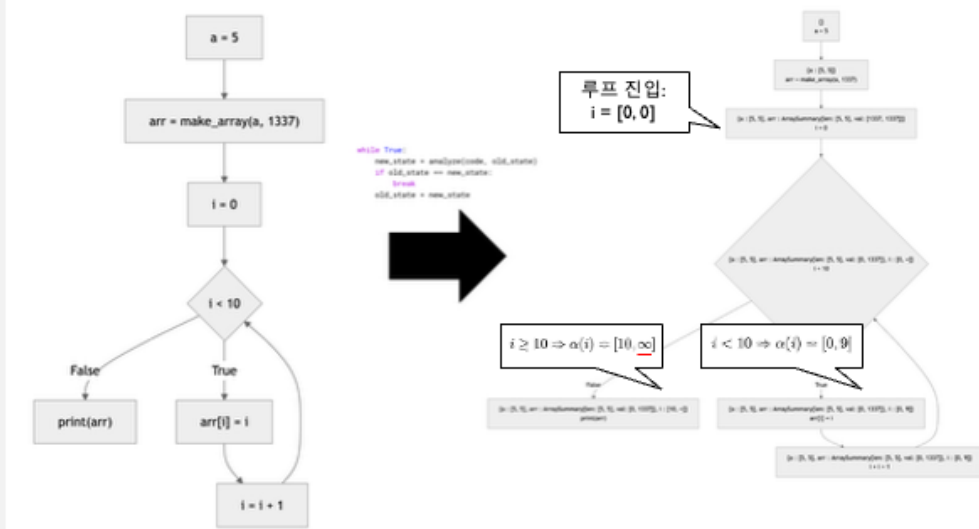


요약도메인 예시: 구간 산술도메인

- 확장된 정수들의 구체적 값들을 구간으로 표현
 - $[n, m] = \{x \mid n \leq x \leq m\}$
 - $\perp$: 어떠한 유효한 값도 존재할 수 없음
 - $\top$: 모든 값이 가능함($= [-\infty, \infty]$)
 - `scanf("%d", a) → a:  $\top$`
- 부분순서관계(partial order)
 - 모든 구간 쌍에 대한 최소상계(lub), 최대하계(glb)가 존재
 - 격자(lattice)로 표현
- 무한한 높이
 - 단순히 구간을 계산할 경우 분석이 종료되지 않음(무한루프)
 - 상계를 활용한 widening 기법을 적용하여 종료 보장



TraceInspector 구조



TracInspector 구조

- 계산된 중간 및 고정점 상태에 대해 오류 조건 검증
- 배열 인덱싱 범위 검사
 - $a[i] \rightarrow 0 \leq i < \text{len}(a)$
- 분기문 조건이 항진/모순명제인지 판별
 - $[P(\sigma)] \neq \top \rightarrow$ 모든 도달 가능 상태 σ 에서 항상 참 또는 거짓
- 오류 발생 위치 및 요약상태를 출력



Type	Function	Line	Message
	InsertionSort	15	Potentially unsafe array indexing: index expr 'j + 1' has value [1, 10], but 'arr.len' has value [10, 10].
	InsertionSort	23	Potentially unsafe array indexing: index expr 'j + 1' has value [0, 10], but 'arr.len' has value [10, 10].

	main	25	Dead branch: boolean expression 'a < b' always evaluates to false
	main	15	Dead branch: boolean expression 'a < b' always evaluates to true
	main	29	Dead branch: boolean expression 'a > 0' always evaluates to true
	main	8	Dead branch: boolean expression 'a ≤ 0' always evaluates to false

TraceInspector 구조

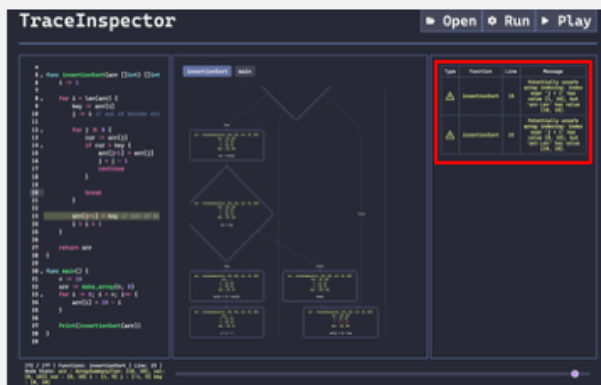


break가 실행될 프로그램 상태 근사



TraceInspector 구조

- 인터페이스 구조: 고정점 계산 흐름을 시각적으로 표현
- CFG와 노드별 요약 상태 표현



탐지된 오류는
우측에 출력

TraceInspector 구조

- 인터페이스 구조: 고정점 계산 흐름을 시각적으로 표현
- CFG와 노드별 요약 상태 표현
- 하단 슬라이더를 통해 고정점 계산 단계별 변화 관찰 가능

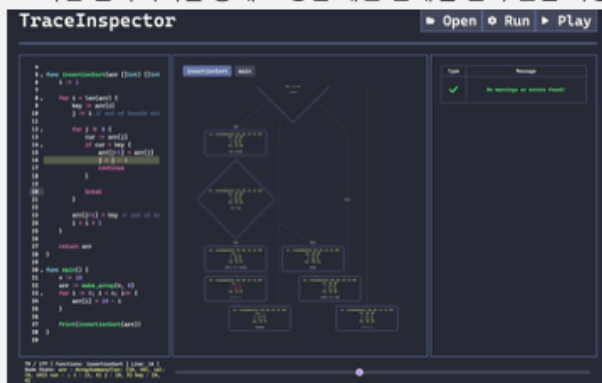


$$f^0 = \perp \rightarrow f^1 = f(\perp) \rightarrow f^2 = f(f(\perp)) \rightarrow f^3 = f(f(f(\perp))) \rightarrow \dots \rightarrow f^{2^p} = f(f^{2^{p-1}})$$



TraceInspector 구조

- 인터페이스 구조: 고정점 계산 흐름을 시각적으로 표현
- CFG와 노드별 요약 상태 표현
- 하단 슬라이더를 통해 고정점 계산 단계별 변화 관찰 가능

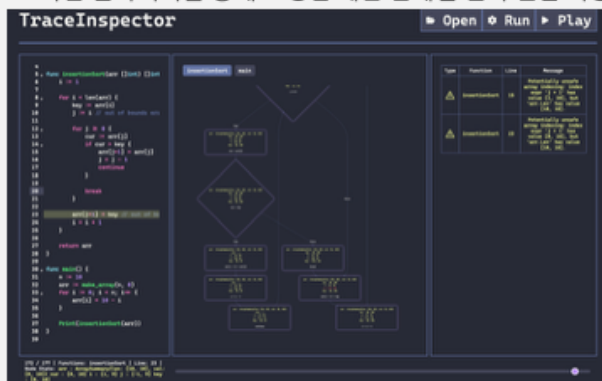


$$f^0 = \perp \rightarrow f^1 = f(\perp) \rightarrow f^2 = f(f(\perp)) \rightarrow f^3 = f(f(f(\perp))) \rightarrow \dots \rightarrow f^{dp} = f(f^{dp})$$



TraceInspector 구조

- 인터페이스 구조: 고정점 계산 흐름을 시각적으로 표현
- CFG와 노드별 요약 상태 표현
- 하단 슬라이더를 통해 고정점 계산 단계별 변화 관찰 가능



$$f^0 = \perp \rightarrow f^1 = f(\perp) \rightarrow f^2 = f(f(\perp)) \rightarrow f^3 = f(f(f(\perp))) \rightarrow \dots \rightarrow f^{dp} = f(f^{dp})$$



한계점 및 향후 방향

- 구간 도메인만을 활용한 근사: $i < \text{len}(\text{arr})$ 과 같은 변수 간 논리적 관계 추론 불가능
 - 분석기 자체는 도메인 인터페이스가 모듈화 되어있음
 - 관계형 도메인 및 reduced product domain을 활용한 정확도 향상
- SimpleProp 체계: $\pm x \odot \pm y \oplus C$ 형태로 표현 가능한 조건문만 정확하게 필터링 가능
 - Difference bound matrix, symbolic algebra, SMT solver 등을 활용하여 일반적 대수식들의 진리값 논증
- ArraySummaryDomain: 배열 전체를 단일 대상으로 간주하기에 정확도 소실
 - Array partition/cell-by-cell domain을 활용한 weak update 지원
- (상호)재귀 함수에 대한 심층적 분석 불가능
 - 현재는 인위적인 호출 스택 높이 제한(이후의 함수 호출은 τ 반환으로 처리)
 - 함수 호출 및 동적 제어 흐름 표현을 위한 interprocedural 분석 구현
- 포인터 및 heap 분석 불가능
 - Abstract heap domain, store/load 구현
- 단일 Go 파일에 대한 분석만 가능
 - Imp를 확장하여 다양한 언어 번역기를 구현
 - 프로젝트 단위 처리 로직 구현

Demonstration

References

- Introduction to Static Analysis: an Abstract Interpretation Perspective, Xavier Rival and Kwangkeun Yi, MIT Press, 2020
- Principles of Abstract Interpretation, Patrick Cousot, MIT Press, 2021
- React-tRace: A Semantics for Understanding React Hooks: An Operational Semantics and a Visualizer for Clarifying React Hooks, J. Lee, et al., Proc. ACM Program. Lang. 9, OOPSLA2 (October 2025). <https://doi.org/10.1145/3763067>
- Static Program Analysis, Anders Møller and Michael I. Schwartzbach, Aarhus University, 2025

Questions?

제 4 장

멘토링 보고서

1차 멘토링(2026-05-14, 강동호)

TraceInspector 캡스톤 팀 단체 멘토링을 강동호 멘토와 진행하였다. 멘토님은 현업 개발자로, 풀스택 개발 및 실제 개발 환경에서의 경험이 많은 분이였다.

우선적으로 우리가 개발한 소프트웨어 시연 및 설명을 진행하였다. 멘토님은 정적분석기가 무엇인지 알고 있었고 실제로 개발 단계에서도 활용하고 있었지만, 우리 소프트웨어만이 가진 장점과 기존에 있는 알고리즘 시각화 웹사이트나 디버거와의 차이점에 대해서도 물어보셨다. 이에 따라 어떤 알고리즘에 의해 분석이 일어나고, 어떤 방식으로 이러한 알고리즘 동작 방식을 보여주고자 했는지 설명을 드렸고 질의응답을 진행했다.

일차적으로 멘토님은 인터페이스상의 보완점들과 캡스톤프로젝트 관점에서 어떤 평가를 받을지에 대해서 말씀하셨다. 예시 코드 목록을 추가하여 사용자가 별도의 준비를 하지 않고도 우리 소프트웨어를 바로 사용해볼 수 있도록 수정하고, 캡스톤 프로젝트에서 큰 이목이나 좋은 평가를 받지 못하더라도 내용적으로 굉장히 깊이가 있고 단지 AI를 활용해서 포장한게 아니라 이 분야에 대해서 제대로 이해하고 구현한 인상을 받았다고 말씀하셨다.

분석기에 대해서도 피드백을 주셨는데, 일차적으로 우리가 Go언어를 분석 대상으로 선정한 이유를 궁금해하면서 많은 사람들이 Go에 대해 크게 관심을 가지지 않을 것을 우려하셨다. 또한 우리 분석기가 여러 파일로 구성된 프로젝트를 분석 가능한지, 그리고 여러 프레임워크들이 통합된 복잡한 소스도 분석 가능하게끔 향후에 개선이 된다면 좋을 것 같다고 말씀하셨다. 멘토님 본인은 이게 실제 개발 환경에서 쓰인다기보단 정적 분석 알고리즘을 시각화하고 이해하는데 많은 도움이 될 것이라고 생각되고, 이런 주제가 참신하고 정적 분석에 대해 배워볼 수 있을 것 같다고 말씀하셨다.

마지막으로 멘토님께 프로젝트 외적으로 실제 개발 현장에서 정적 분석 도구 등 프로그래밍 언어론과 관련된 도구나 방법을 응용하고 있는지 물어보았고, 실제 개발자의 입장에서 어떤 방식으로 소프트웨어를 분석하는 도구가 필요한지 여쭙보았다. 멘토님은 보안과 관련된 문제들이 굉장히 민감하다면서 이러한 취약점들을 분석해주는 분석기가 더 발전되면 유용할 것 같다고 말씀하셨고, LLM 역시 코드를 많이 생성하니 이러한 코드들도 신뢰할 수 있게 분석이나 검증이 가능하다면 좋을 것 같다고 말씀하셨다.

이번 멘토링을 통해 피드백은 물론, 스스로 어떻게 정적 분석이라는 개념을 남들에게 잘 설명할 수 있을까에 대해 고민을 하게 되었다. 이미 해당 분야를 들어보고 프로그래밍과 익숙한 멘토님께도 우리 정적분석기가 정확히 코드 시각화 서비스나 디버거랑 어떤 방면에 차이가 있는지 와닿을 수 있는 방식으로 설명하는게 어렵다는 것을 이번 기회를 통해 알게 되었다. 따라서 최종발표 준비를 할 때 이런 관점에서 설명을 준비하고 우리가 구현한 소프트웨어의 기능들과 한계들을 어떻게 잘 나타낼 수 있을지에 대해 다시금 생각해보게 되었다.

2차 멘토링(2026-05-17, 이광현)

TraceInspector 팀 멘토링을 이광현 멘토와 진행하였다. 멘토링을 시작하기에 앞서, 멘토링 페이지에 게시된 멘토의 설명과 실제 멘토가 현재 진행중인 분야가 완전히 다름을 멘토가 언급했다. 하드웨어 설계보다는 AI모형 만드는 업무를 하고 있고, 이에 따라 멘토링 페이지에 본인 링크드인 등을 첨부하기를 희망한다는 얘기를 하였다.

이번 멘토링에서는 최종 발표자료가 준비되었던 관계로 멘토에게 우리 프로젝트 발표와 시연을 먼저 보여주었다. 멘토는 우리의 발표를 보고 다음과 같은 질문들을 하였다:

1. 프로그램 실행 흔적 의미론을 설명하는 과정에서 테스트는 유한한 갯수의 흔적만을 볼 수 있다고 나와있는데, 테스트 coverage와 어떤 연관이 있는지?
2. 이러한 요약해석 기법과 테스트와 관계가 어떻게 되고 어떤 식으로 서로 활용할 수 있는지?
3. Go나 C의 경우 정수 표현 범위가 정해져 있는데, 정적 분석기에서는 이러한 의미를 반영하는지?

이처럼 우리 분석기 뿐만 아니라 정적 분석 자체에 대해 다양한 질문을 해주셨고, 특히 정적 분석기를 시각적으로 보여주는 인터페이스는 정말 참신한 아이디어이며 처음 본다는 말씀을 해주셨다.

하지만 이전 멘토링과 마찬가지로, 실제로 가지고 있는 기술적 깊이와 구현된 분석기의 완성도와는 별개로 사용자와 평가자들이 보는 인터페이스 상에서 한눈에 분석 결과가 어떤 식으로 나오는지 보여주는 핵심적인 강조점이 잘 안보인다고 피드백을 하였다. 이에 실제로 오류가 검출되는 상태에서 문제가 되는 변수의 상태 등을 강조하여 시각적으로 돋보일 수 있다면 더 좋을거 같다고 조언하였고, 프로젝트의 방향 자체를 실제로 개발에서 많은 유저들이 사용하는 방향보다는 정적 분석이라는 분야에 대해 깊이 탐구하고 교육적 목표의 서비스를 구현했다는 점을 강조하는 방향이 더 유리할 것이라는 피드백을 주었다.

제 5 장

회의록

1차 주간 회의 (2026.03.06)

첫 번째 팀 회의에서는 프로젝트 주제의 구체화와 역할 분담에 대해 논의하였다. 프로젝트의 목표를 Go 언어로 작성된 프로그램을 정적으로 분석하여 잠재적인 런타임 오류를 탐지하고 이를 사용자에게 제공하는 서비스로 정의하였다. 또한 사용자가 소스 코드를 업로드하고 분석 결과를 확인할 수 있도록 웹 기반 인터페이스를 구축하기로 결정하였으며, 인터페이스는 입력 및 업로드 영역과 분석 결과 시각화 영역으로 구성하기로 하였다.

역할 분담에 있어서는 분석기 이론 및 핵심 분석 로직 구현과 웹 인터페이스 개발을 분리하여 진행하기로 하였다. 이에 따라 김신영은 요약해석 기반 분석기의 의미론 설계 및 구현을 담당하고, 강민성은 프론트엔드 및 서비스 환경 구축을 담당하기로 결정하였다. 또한 다음 회의 전까지 Go 언어의 지원 범위를 정의하고 언어 요소별 요약 의미론의 초안을 설계하며, 동시에 웹 인터페이스의 기본 구조를 구현하기로 하였다.

2차 주간 회의 (2026.03.19)

두 번째 팀 회의에서는 분석기와 프론트엔드 간의 인터페이스를 구체화하고, 제어 흐름 그래프(Control Flow Graph, CFG)의 출력 형식과 데이터 교환 방식을 확정하였다. 프론트엔드 구현 기술로 기존 JavaScript 기반 계획 대신 C# Razor 프레임워크를 사용하기로 결정하였으며, 코드 입력 영역과 CFG 시각화 영역을 분리하는 사용자 인터페이스 구성을 논의하였다. 또한 분석기와 웹 서비스 간의 통신은 JSON 기반 인터페이스를 통해 수행하기로 결정하였다.

분석기 측에서는 Go 언어에 대한 요약해석 기반 정적 분석기의 설계를 위해 관련 문헌을 조사하고 언어 요소별 의미론 정의를 위한 기초 연구를 진행하였다. 구현 측면에서는 CFG 생성을 위한 Go 추상 구문 트리 순회 기능의 초안을 작성하였으며, 프로젝트 저장소를 개설하고 분석기와 프론트엔드 간의 입출력 명세를 확정하였다. 특히 CFG 출력, 분석 과정의 중간 상태 갱신 정보, 최종 오류 탐지 결과를 표준화된 형식으로 교환하기로 결정하여 향후 양 모듈의 독립적인 개발과 통합이 가능하도록 기반을 마련하였다.

3차 주간 회의 (2026.03.24)

세 번째 팀 회의에서는 프론트엔드 인터페이스 초안에 대한 검토와 제어 흐름 그래프(Control Flow Graph, CFG) 생성 기능의 구현 현황을 점검하였다. 프론트엔드 측에서는 코드 입력 영역, 그래프 시각화 영역, 로그 출력 영역으로 구성된 초기 사용자 인터페이스 레이아웃을 제시하였으며, 향후 텍스트 에디터와 그래프 시각화 기능을 통합하기 위한 구체적인 구현 방향을 논의하였다. 또한 사용자 경험 측면에서 코드 작성, 분석 실행, 결과 확인이 하나의 화면에서 이루어질 수 있도록 인터페이스 구조를 정교화하기로 하였다.

분석기 측에서는 Go 프로그램으로부터 제어 흐름 그래프를 생성하는 기능의 구현을

완료하고, 간단한 예제 프로그램을 대상으로 정상적으로 동작함을 확인하였다. 아울러 요약해석 기반 정적 분석을 위한 이론적 설계를 지속적으로 진행하였으며, 분석 목표와 구현 난이도를 고려하여 적절한 요약 도메인을 검토하였다. 회의 결과 현재 분석기의 핵심 기반인 CFG 생성 단계는 완료된 것으로 판단하였으며, 향후에는 프로그램 상태를 표현하기 위한 요약 도메인과 의미론을 정립하고 분석 기능 구현에 집중하기로 결정하였다. 또한 Interval 도메인과 프로그램-레이블 쌍 집합 기반의 추상화 도메인을 우선적인 구현 대상으로 선정하였다.

4차 주간 회의 (2026.04.02)

네 번째 팀 회의에서는 프론트엔드 및 분석기 구현 현황을 점검하고, 중간 발표 준비를 위한 향후 개발 계획을 수립하였다. 프론트엔드 측에서는 기존 HTML/CSS 기반 시제품을 TypeScript 및 React 기반 구조로 전환하는 작업을 완료하였으며, 코드 입력 기능을 위해 CodeMirror 에디터를 도입하고 그래프 시각화를 위해 Mermaid.js를 활용하기로 결정하였다. 이를 통해 향후 코드 편집, 분석 실행, 결과 시각화 기능을 통합한 웹 인터페이스를 구축할 수 있는 기반을 마련하였다.

분석기 측에서는 Go 언어의 추상적 의미론 설계를 지속적으로 수행하였으며, 반복문에 대한 widening 연산과 조건문 분기 처리 방안을 구체화하였다. 또한 구간(Interval) 추상도메인을 구현하고, 추상도메인을 모듈화된 인터페이스 형태로 설계하여 향후 다양한 추상도메인을 손쉽게 추가할 수 있도록 구조를 개선하였다. 회의에서는 현재까지 CFG 생성 단계와 구간도메인 구현이 완료되었음을 확인하였으며, 다음 단계로 추상 의미론의 실제 구현과 CFG 상에서의 기호 실행(symbolic execution)을 수행하기 위한 드라이버 코드를 개발하기로 결정하였다.

추가적으로 중간 발표 준비를 위해 현재까지의 구현 성과를 정리하고, 분석기와 프론트엔드의 핵심 기능을 시연할 수 있는 데모 환경을 구축하는 방안을 논의하였다. 향후에는 추상 의미론 구현을 완료하고, 웹 인터페이스에 텍스트 에디터 및 그래프 시각화 기능을 통합하는 작업을 중점적으로 진행하기로 하였다.

5차 주간 회의 (2026.04.06)

다섯 번째 팀 회의에서는 중간 발표 준비와 함께 프론트엔드 및 분석기 구현 현황을 점검하였다. 프론트엔드 측에서는 CodeMirror 기반 코드 편집기를 인터페이스에 통합하였으며, 제어 흐름 그래프(CFG) 정보를 해석하고 시각화하기 위한 기능 구현에 착수하였다. 또한 현재까지의 개발 결과와 향후 인터페이스 확장 계획을 정리하여 중간 발표 자료 제작을 진행하였다.

분석기 측에서는 Imp 언어의 의미론 정의와 Go 프로그램을 Imp 중간 표현으로 변환하

는 번역기를 구현하였으며, Imp 프로그램에 대한 CFG 생성 알고리즘을 완성하였다. 이를 통해 코드 전처리 단계가 완료되었으며, 이후에는 설계한 요약 의미론을 기반으로 실제 요약해석 프레임워크와 분석 알고리즘 구현에 집중하기로 하였다. 또한 중간 발표에서는 분석기와 프론트엔드 영역을 분리하여 각각 이론적 배경, 구현 현황, 인터페이스 설계 및 향후 개발 계획을 소개하기로 결정하였다.

6차 주간 회의 (2026.04.16)

여섯 번째 팀 회의에서는 중간 발표 자료를 최종 검토하고 시연 영상을 촬영하는 한편, 분석기와 프론트엔드 간의 인터페이스를 보완하기 위한 논의를 진행하였다. 프론트엔드 측에서는 Go 언어 구문 강조 기능을 코드 에디터에 적용하였으며, Go 소스 파일을 직접 불러와 편집기에 표시하는 기능을 구현하였다. 또한 그래프 시각화와 백엔드 서버 구축을 위해 Express 기반 환경을 도입하여 통합 작업을 진행하였다.

분석기 측에서는 함수 단위 분석을 지원하기 위해 CFG 노드 식별 체계를 개선하고, 함수 이름과 연계된 고유 식별자를 생성하도록 구조를 수정하였다. 이에 따라 JSON 출력 형식도 함께 갱신되었으며, 고정점 계산 프레임워크와 요약 도메인 구현을 본격적으로 도입하기 시작하였다. 회의에서는 중간 발표 자료와 시연 영상을 최종 점검하였고, 함수별 CFG 출력과 분석 중간 결과의 교환 형식을 확정하여 프론트엔드와 분석기 간의 연동 기반을 강화하였다.

7차 주간 회의 (2026.04.30)

일곱 번째 팀 회의에서는 분석기 출력 결과를 검토하고 프론트엔드의 그래프 시각화 결과에 대한 피드백을 진행하였다. 프론트엔드 측에서는 그래프 렌더링 기능과 탭 인터페이스를 개선하였으며, 시연 환경의 안정성을 확보하기 위해 별도의 웹 서버를 구축하였다. 또한 분석기에서 제공하는 디버그 출력 기능을 활용하기 위한 결과 출력 영역 구현을 시작하였다.

분석기 측에서는 반복문을 포함한 언어의 주요 구성 요소를 처리할 수 있는 첫 번째 분석기 버전을 완성하였으며, 삽입 정렬과 같은 예제 프로그램을 대상으로 분석 결과를 검증하였다. 특히 고정된 크기의 배열에 대한 참조 범위 오류를 성공적으로 탐지함으로써 분석기의 기본 기능이 정상적으로 동작함을 확인하였다. 또한 분석 과정 중 발생하는 상태 변화 정보를 프론트엔드와 공유하기 위한 출력 형식을 정의하고, 이에 맞추어 상태 갱신 이벤트를 생성하도록 구현하였다. 회의에서는 정보, 경고, 오류 메시지 및 상태 갱신을 위한 JSON 통신 규격을 확정하고 향후 사용자 인터페이스 개선 방향을 함께 논의하였다.

8차 주간 회의 (2026.05.07)

여덟 번째 팀 회의에서는 분석기와 프론트엔드 간 상태 출력 형식을 표준화하고, 양 시스템의 연동 상태를 점검하였다. 프론트엔드 측에서는 상태 출력 창을 구현하는 과정에서 사용자 인터페이스를 전반적으로 개선하였으며, 창 여백 제거, 스크롤 동작 개선, Upload 및 Run 기능 통합, 탭 관련 버그 수정 등 다수의 사용성 개선 작업을 수행하였다.

분석기 측에서는 디버그 출력 기능과 도메인 연산 결과를 선택적으로 활성화할 수 있도록 수정하여, 실제 실행 시에는 JSON 기반 결과만 출력하도록 구조를 개선하였다. 또한 프론트엔드와의 교차 검증을 통해 출력 형식을 지속적으로 조정하고 호환성을 검증하였다. 회의 결과 상태 출력 창, 하이라이트 기능, 상태 변화 탐색을 위한 슬라이더 기능을 포함한 최종 인터페이스 구현 방향을 확정하였으며, 포스터 초안 및 최종보고서 작성을 위한 준비 작업을 병행하기로 하였다.

9차 주간 회의 (2026.05.14)

아홉 번째 팀 회의에서는 분석기의 요약 도메인 관련 오류 수정과 프론트엔드 인터페이스 개선 작업을 점검하고, 프로젝트 결과물 정리를 위한 포스터 및 발표 자료 작성 계획을 논의하였다. 프론트엔드 측에서는 사용자 인터페이스를 전반적으로 개편하여 색상 테마를 Nord에서 Dracula로 변경하였으며, 상태 변화가 보다 명확하게 드러나도록 그래프 출력 형식을 개선하였다. 또한 분석 과정의 변화를 시간 순서대로 확인할 수 있도록 타임라인 슬라이더를 추가하고, 다양한 시각적 효과와 하이라이트 기능을 보강하였다.

분석기 측에서는 구간 추상 도메인과 확장 정수(extended integer) 연산에서 발견된 건전성 문제를 수정하였으며, 고정점 계산 이후의 상태를 활용하는 post-fixpoint analysis를 구현하였다. 이를 통해 반복문 및 조건문 가드가 항상 참 또는 거짓으로 평가되는 경우를 탐지하여 경고를 생성하는 기능을 추가하였다. 회의에서는 포스터 작성을 완료하여 제출하였고, 최종 발표 자료와 요약 결과 보고서 작성을 위한 마무리 작업 계획을 수립하였다.

주 의 문

2026년 다학제간캡스톤디자인I 최종보고서
TraceInspector

발행인 : 김신영, 강민성

발행일 : 2026년 6월 19일

발행처 : 국민대학교 소프트웨어융합대학 소프트웨어학부

주소 : (136-702) 서울시 성북구 정릉동 861-1

1. 이 보고서는 국민대학교 소프트웨어융합대학
소프트웨어학부에서 개설한 교과목
다학제간캡스톤디자인I 에서 수행한 프로젝트
최종보고서 입니다.
 2. 이 보고서의 내용은 국민대학교 소프트웨어학부 및 위
발행인들의 서면 허락없이 사용되거나, 재가공 될 수
없습니다.
-